

INDEX

Sr. no.	Name of the practical	Page No.	Date	Sign
1	Installing and setting environment variables for Working with Apache Hadoop	1-12		
2	Implementing Map-Reduce Program for Word Count problem,	13-21		
3	Write a program to Implement a tri-gram model	22-23		
4	Download and install Spark. Create Graphical data and access the graphical data using Spark	24-27		
5	Write a Spark code for the given application and handle error and recovery of data	28-33		
6	Write a Spark code to Handle the Streaming of data.	34-39		
7	Install HBase and use the HBase Data model Store and retrieve data	40-48		
8	Perform importing and exporting of data between SQL and Hadoop using Sqoop.	49-51		

Practical No - 1

Aim: Installing and setting environment variables for Working with Apache Hadoop

Hadoop Installation in Ubuntu

Introduction

Apache Hadoop is an open-source software framework used to store, manage and process large datasets for various big data computing applications running under clustered systems. It is Javabased and uses Hadoop Distributed File System (HDFS) to store its data and process data using MapReduce. In this article, you will learn how to install and configure Apache Hadoop on Ubuntu

Install Java

sudo apt install openjdk-8-jre

```
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

mohit@mohit-virtual-machine:~$ sudo apt install openjdk-8-jre
[sudo] password for mohit:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ca-certificates-java fonts-dejavu-extra java-common libatk-wrapper-java
  libatk-wrapper-java-jni openjdk-8-jre-headless
```

sudo apt install openjdk-8-jdk

```
Processing triggers for man-db (2.10.2-1) ...
mohit@mohit-virtual-machine:~$ sudo apt install openjdk-8-jdk
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
```

Verify the installed version of Java.

java -version

```
Setting up libx11-dev:amd64 (2:1.7.5-1ubuntu0.3) ...
Setting up libxt-dev:amd64 (1:1.2.1-1) ...
mohit@mohit-virtual-machine:~$ java -version
openjdk version "1.8.0_452"
OpenJDK Runtime Environment (build 1.8.0_452-8u452-ga-us1-0ubuntu1~22.04-b09)
OpenJDK 64-Bit Server VM (build 25.452-b09, mixed mode)
```

Create Hadoop User and Configure Password-less SSH

Add a new user hadoop. sudo adduser hadoop

```

mohit@mohit-virtual-machine:~$ sudo adduser hadoop
Adding user `hadoop' ...
Adding new group `hadoop' (1001) ...
Adding new user `hadoop' (1001) with group `hadoop' ...
Creating home directory `/home/hadoop' ...
Copying files from `/etc/skel' ...

```

Add the hadoop user to the sudo group.

```

mohit@mohit-virtual-machine:~$ sudo usermod -aG sudo hadoop
mohit@mohit-virtual-machine:~$

```

Install the OpenSSH server and client.

```

mohit@mohit-virtual-machine:~$ sudo apt install openssh-server openssh-client -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done

```

When you get a prompt, respond with: keep the local version currently installed

Log in with hadoop user. `sudo su - hadoop`

```

mohit@mohit-virtual-machine:~$ sudo su - hadoop
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

hadoop@mohit-virtual-machine:~$

```

Generate public and private key pairs. `ssh-keygen`

`-t rsa`

(don't add password just press enter as many times to reach normal prompt)

```

hadoop@mohit-virtual-machine:~$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/home/hadoop/.ssh/id_rsa):
Created directory '/home/hadoop/.ssh'.
Enter passphrase (empty for no passphrase):

```

Add the generated public key from `id_rsa.pub` to `authorized_keys`.

`sudo cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys`

```

+-----[SHA256]-----+
hadoop@mohit-virtual-machine:~$ sudo cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
[sudo] password for hadoop:
hadoop@mohit-virtual-machine:~$

```

Change the permissions of the `authorized_keys` file.

`sudo chmod 640 ~/.ssh/authorized_keys`

```

hadoop@mohit-virtual-machine:~$ sudo chmod 640 ~/.ssh/authorized_keys
hadoop@mohit-virtual-machine:~$

```

Verify if the password-less SSH is functional.

ssh localhost

```
hadoop@mohit-virtual-machine:~$ ssh localhost
```

Install Apache Hadoop

Download the latest stable version of Hadoop. To get the latest version, go to Apache Hadoop official download page. wget <https://archive.apache.org/dist/hadoop/core/hadoop-3.1.1/hadoop-3.1.1.tar.gz>

```
hadoop@mohit-virtual-machine:~$ wget https://archive.apache.org/dist/hadoop/core/hadoop-3.1.1/hadoop-3.1.1.tar.gz
--2025-06-10 09:52:49-- https://archive.apache.org/dist/hadoop/core/hadoop-3.1.1/hadoop-3.1.1.tar.gz
```

Extract the downloaded file.

tar -xvzf hadoop-3.1.1.tar.gz

```
hadoop-3.1.1/share/doc/hadoop/hadoop-client-runtime/css/site.css
hadoop-3.1.1/share/doc/hadoop/hadoop-client-runtime/css/maven-theme.css
hadoop-3.1.1/share/doc/hadoop/hadoop-client-runtime/css/print.css
hadoop-3.1.1/share/doc/hadoop/hadoop-client-runtime/dependency-analysis.html
```

Move the extracted directory to the /usr/local/ directory.

sudo mv hadoop-3.1.1 /usr/local/hadoop

```
hadoop-3.1.1/share/doc/hadoop/hadoop-client-runtime/css/print.css
hadoop-3.1.1/share/doc/hadoop/hadoop-client-runtime/dependency-analysis.html
hadoop@mohit-virtual-machine:~$ sudo mv hadoop-3.1.1 /usr/local/hadoop
hadoop@mohit-virtual-machine:~$
```

Create directory to store system logs.

sudo mkdir /usr/local/hadoop/logs

```
hadoop@mohit-virtual-machine:~$ sudo mkdir /usr/local/hadoop/logs
hadoop@mohit-virtual-machine:~$
```

Change the ownership of the hadoop directory.

sudo chown -R hadoop:hadoop /usr/local/hadoop

```
hadoop@mohit-virtual-machine:~$ sudo chown -R hadoop:hadoop /usr/local/hadoop
hadoop@mohit-virtual-machine:~$
```

Configure Hadoop

Edit file ~/.bashrc to configure the Hadoop environment variables.

```
hadoop@mohit-virtual-machine:~$ sudo nano ~/.bashrc
hadoop@mohit-virtual-machine:~$
```


Add the following lines to the file. Save and close the file.

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_HOME=/usr/local/hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export
HADOOP_MAPRED_HOME=$HADOOP_HOME
E export
HADOOP_COMMON_HOME=$HADOOP_HOME
ME export
HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME export
HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native export
PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"
```

```
hadoop@mohit-virtual-machine: ~
GNU nano 6.2 /home/hadoop/.bashrc
fi

export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_HOME=/usr/local/hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"

[ Wrote 135 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut      ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste    ^J Justify  ^_ Go To Line
```

Activate the environment variables.

```
source ~/.bashrc
```

```
hadoop@mohit-virtual-machine:~$ source ~/.bashrc
hadoop@mohit-virtual-machine:~$
```

Configure Java Environment Variables

Hadoop has a lot of components that enable it to perform its core functions. To configure these components such as YARN, HDFS, MapReduce, and Hadoop-related project settings, you need to define Java environment variables in `hadoop-env.sh` configuration file.

Find the Java path.

which javac

```
hadoop@mohit-virtual-machine:~$ which javac
/usr/bin/javac
```

Find the OpenJDK directory.

readlink -f /usr/bin/javac

```
hadoop@mohit-virtual-machine:~$ readlink -f /usr/bin/javac
/usr/lib/jvm/java-8-openjdk-amd64/bin/javac
```

Edit the `hadoop-env.sh` file. `sudo nano`

`$HADOOP_HOME/etc/hadoop/hadoop-env.sh`

```
hadoop@mohit-virtual-machine:~$ sudo nano $HADOOP_HOME/etc/hadoop/hadoop-env.sh
hadoop@mohit-virtual-machine:~$
```

Add the following lines to the file. Then, close and save the file.

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64 export
HADOOP_CLASSPATH+=" $HADOOP_HOME/lib/*.*jar"
```

```
hadoop@mohit-virtual-machine: ~
GNU nano 6.2 /usr/local/hadoop/etc/hadoop/hadoop-env.sh *
###
# Advanced Users Only!
###
#
# When building Hadoop, one can add the class paths to the commands
# via this special env var:
# export HADOOP_ENABLE_BUILD_PATHS="true"
#
# To prevent accidents, shell commands be (superficially) locked
# to only allow certain users to execute certain subcommands.
# It uses the format of (command)_(subcommand)_USER.
#
# For example, to limit who can execute the namenode command,
# export HDFS_NAMENODE_USER=hdfs

export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_CLASSPATH+=" $HADOOP_HOME/lib/*.*jar"
[ Wrote 427 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut      ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste    ^J Justify  ^_ Go To Line
```

`OOP_HOME/lib/*.*jar"`

Browse to the hadoop lib directory.

cd /usr/local/hadoop/lib

```
hadoop@mohit-virtual-machine:~$ cd /usr/local/hadoop/lib
hadoop@mohit-virtual-machine:/usr/local/hadoop/lib$
```

Download the Javax activation file. sudo wget

<https://jcenter.bintray.com/javax/activation/javax.activation-api/1.2.0/javax.activation-api-1.2.0.jar>

```
hadoop@mohit-virtual-machine:/usr/local/hadoop/lib$ sudo wget https://jcenter.b
intray.com/javax/activation/javax.activation-api/1.2.0/javax.activation-api-1.2
.0.jar
--2025-06-10 10:15:59-- https://jcenter.bintray.com/javax/activation/javax.act
ivation-api/1.2.0/javax.activation-api-1.2.0.jar
Resolving jcenter.bintray.com (jcenter.bintray.com)... 18.214.194.113, 18.232.1
72.199, 3.95.117.170
Connecting to jcenter.bintray.com (jcenter.bintray.com)|18.214.194.113|:443...
connected.
```

Come back to home folder of Hadoop user

cd /home/hadoop

```
hadoop@mohit-virtual-machine:/usr/local/hadoop/lib$ cd /home/hadoop
hadoop@mohit-virtual-machine:~$
```

Verify the Hadoop version.

hadoop version

```
hadoop@mohit-virtual-machine:~$ hadoop version
Hadoop 3.1.1
Source code repository https://github.com/apache/hadoop -r 2b9a8c1d3a2caf1e733d
57f346af3ff0d5ba529c
Compiled by leftnoteasy on 2018-08-02T04:26Z
Compiled with protoc 2.5.0
From source with checksum f76ac55e5b5ff0382a9f7df36a3ca5a0
This command was run using /usr/local/hadoop/share/hadoop/common/hadoop-common-
3.1.1.jar
hadoop@mohit-virtual-machine:~$
```

Edit the core-site.xml configuration file to specify the URL for your NameNode.

sudo nano \$HADOOP_HOME/etc/hadoop/core-site.xml

```
hadoop@mohit-virtual-machine:~$ sudo nano $HADOOP_HOME/etc/hadoop/core-site.xml
hadoop@mohit-virtual-machine:~$
```

Add the following lines. Save and close the file.

<configuration>

<property>

<name>fs.default.name</name>

<value>hdfs://0.0.0.0:9000</value>

<description>The default file system URI</description>

```
</property>
</configuration>
```

```
<!-- Put site-specific property overrides in this file. -->

<configuration>
<property>
<name>fs.default.name</name>
<value>hdfs://0.0.0.0:9000</value>
<description>The default file system URI</description>
</property>
</configuration>
```

[Wrote 25 lines]

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify

Create a directory for storing node metadata and change the ownership to hadoop.

```
sudo mkdir -p /home/hadoop/hdfs/{namenode,datanode} sudo chown -R
```

```
hadoop:hadoop /home/hadoop/hdfs
```

```
hadoop@mohit-virtual-machine:~$ sudo mkdir -p /home/hadoop/hdfs/{namenode,datanode}
hadoop@mohit-virtual-machine:~$ sudo chown -R hadoop:hadoop /home/hadoop/hdfs
hadoop@mohit-virtual-machine:~$
```

Edit hdfs-site.xml configuration file to define the location for storing node metadata, fs-image file.

```
sudo nano $HADOOP_HOME/etc/hadoop/hdfs-site.xml
```

```
hadoop@mohit-virtual-machine:~$ 
hadoop@mohit-virtual-machine:~$ sudo nano $HADOOP_HOME/etc/hadoop/hdfs-site.xml
hadoop@mohit-virtual-machine:~$
```

Add the following lines. Close and save the file.

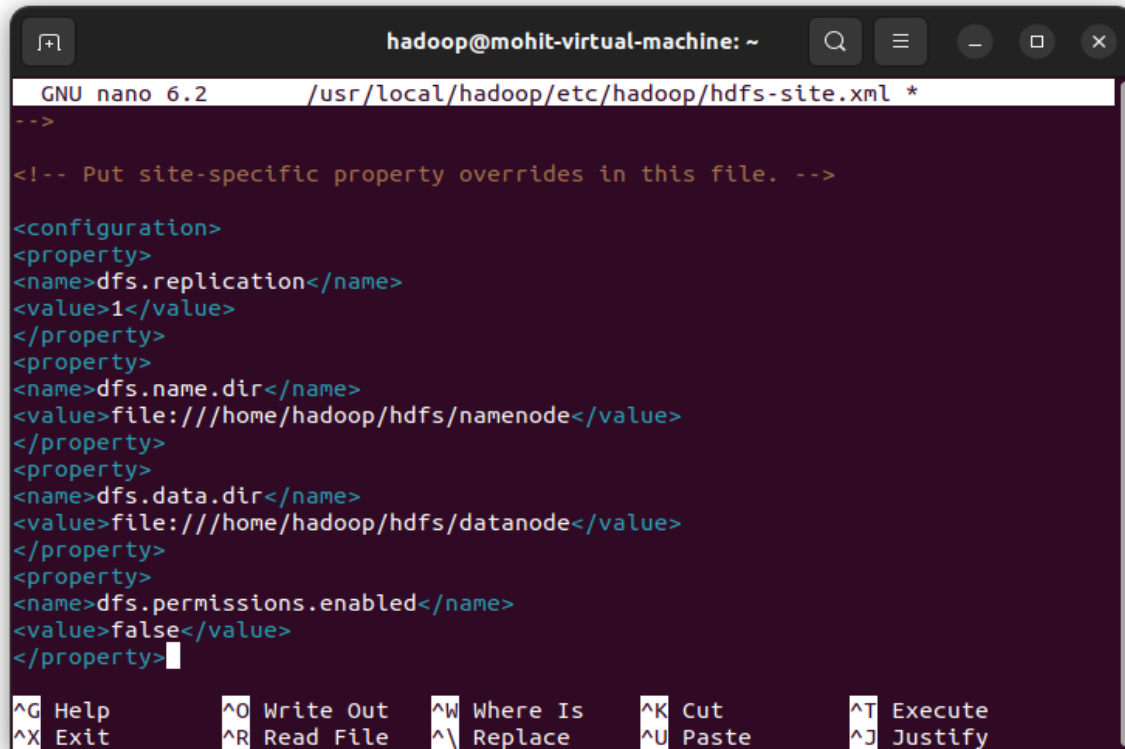
```
<configuration>
<property>
<name>dfs.replication</name>
<value>1</value>
</property>
<property>
<name>dfs.name.dir</name>
<value>file:///home/hadoop/hdfs/namenode</value>
</property>
<property>
<name>dfs.data.dir</name>
<value>file:///home/hadoop/hdfs/datanode</value>
</property>
<property>
```



```

<name>dfs.permissions.enabled</name>
<value>>false</value>
</property>
</configuration>

```



```

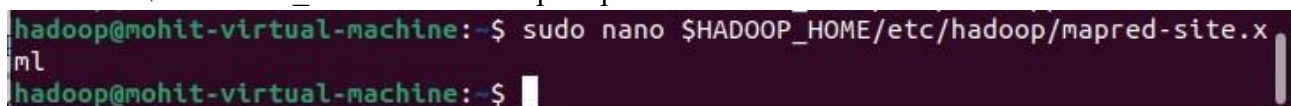
hadoop@mohit-virtual-machine: ~
GNU nano 6.2 /usr/local/hadoop/etc/hadoop/hdfs-site.xml *
-->
<!-- Put site-specific property overrides in this file. -->

<configuration>
<property>
<name>dfs.replication</name>
<value>1</value>
</property>
<property>
<name>dfs.name.dir</name>
<value>file:///home/hadoop/hdfs/namenode</value>
</property>
<property>
<name>dfs.data.dir</name>
<value>file:///home/hadoop/hdfs/datanode</value>
</property>
<property>
<name>dfs.permissions.enabled</name>
<value>>false</value>
</property>

```

Edit mapred-site.xml configuration file to define MapReduce values.

```
sudo nano $HADOOP_HOME/etc/hadoop/mapred-site.xml
```



```

hadoop@mohit-virtual-machine:~$ sudo nano $HADOOP_HOME/etc/hadoop/mapred-site.x
ml
hadoop@mohit-virtual-machine:~$

```

Add the following lines. Save and close the file.

```

<configuration>
<property>
<name>mapreduce.framework.name</name>
<value>yarn</value>
</property>
<property>
<name>yarn.app.mapreduce.am.env</name>
<value>HADOOP_MAPRED_HOME=/usr/local/hadoop</value>
<description>Change this to your hadoop location.</description>
</property>
<property>

```

```

<name>mapreduce.map.env</name>
<value>HADOOP_MAPRED_HOME=/usr/local/hadoop</value>
<description>Change this to your hadoop location.</description>
</property>
<property>
<name>mapreduce.reduce.env</name>
<value>HADOOP_MAPRED_HOME=/usr/local/hadoop</value>
<description>Change this to your hadoop location.</description>
</property>
</configuration>

```

```

GNU nano 6.2 /usr/local/hadoop/etc/hadoop/mapred-site.xml
<property>
<name>mapreduce.framework.name</name>
<value>yarn</value>
</property>
<property>
<name>yarn.app.mapreduce.am.env</name>
<value>HADOOP_MAPRED_HOME=/usr/local/hadoop</value>
<description>Change this to your hadoop location.</description>
</property>
<property>
<name>mapreduce.map.env</name>
<value>HADOOP_MAPRED_HOME=/usr/local/hadoop</value>
<description>Change this to your hadoop location.</description>
</property>
<property>
<name>mapreduce.reduce.env</name>
<value>HADOOP_MAPRED_HOME=/usr/local/hadoop</value>
<description>Change this to your hadoop location.</description>
</property>
</configuration>
[ Wrote 39 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify

```

Edit the yarn-site.xml configuration file and define YARN-related settings.

```
sudo nano $HADOOP_HOME/etc/hadoop/yarn-site.xml
```

```

hadoop@mohit-virtual-machine:~$ sudo nano $HADOOP_HOME/etc/hadoop/yarn-site.xml
hadoop@mohit-virtual-machine:~$

```

Add the following lines. Save and close the file.

```

<configuration>
<property>
<name>yarn.nodemanager.aux-services</name>
<value>mapreduce_shuffle</value>
</property>
</configuration>

```

```
-->
<configuration>

<!-- Site specific YARN configuration properties -->
<property>
<name>yarn.nodemanager.aux-services</name>
<value>mapreduce_shuffle</value>
</property>

[ Wrote 22 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Log in with hadoop user.(if you are not. This will be case once you have restarted you have computer) `sudo su – hadoop`

```
hadoop@mohit-virtual-machine:~$ sudo su - hadoop
hadoop@mohit-virtual-machine:~$
```

Validate the Hadoop configuration and format the HDFS NameNode.

`hdfs namenode -format`

```
hadoop@mohit-virtual-machine:~$ hdfs namenode -format
2025-06-10 10:42:12,193 INFO namenode.NameNode: STARTUP_MSG:
/*****
STARTUP_MSG: Starting NameNode
STARTUP_MSG:  host = mohit-virtual-machine/127.0.1.1
STARTUP_MSG:  args = [-format]
STARTUP_MSG:  version = 3.1.1
STARTUP_MSG:  ...
```

Start the Apache Hadoop Cluster

Start the NameNode and DataNode. `start-dfs.sh`

```
hadoop@mohit-virtual-machine:~$ start-dfs.sh
Starting namenodes on [0.0.0.0]
0.0.0.0: Warning: Permanently added '0.0.0.0' (ED25519) to the list of known hosts.
Starting datanodes
Starting secondary namenodes [mohit-virtual-machine]
mohit-virtual-machine: Warning: Permanently added 'mohit-virtual-machine' (ED25519) to the list of known hosts.
hadoop@mohit-virtual-machine:~$
```

Start the YARN resource and node managers.

`start-yarn.sh`

```
hadoop@mohit-virtual-machine:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
hadoop@mohit-virtual-machine:~$
```

Verify all the running components.

`Jps`

```

hadoop@mohit-virtual-machine:~$ jps
12885 NodeManager
12533 ResourceManager
12021 DataNode
12204 SecondaryNameNode
11869 NameNode
13071 Jps
hadoop@mohit-virtual-machine:~$

```

Access Apache Hadoop Web Interface

You can access the Hadoop NameNode and DataNode on your browser via <http://localhost:9870>.

For example:

<http://localhost:9870>

The screenshot shows the Hadoop NameNode Web Interface. The browser address bar displays `localhost:9870/dfshealth.html#tab-overview`. The page has a green navigation bar with tabs: Hadoop, Overview, Datanodes, Datanode Volume Failures, Snapshot, Startup Progress, and Utilities. The main content area is titled "Overview '0.0.0.0:9000' (active)". Below the title is a table with the following information:

Started:	Tue Jun 10 10:44:02 +0530 2025
Version:	3.1.1, r2b9a8c1d3a2caf1e733d57f346af3ff0d5ba529c
Compiled:	Thu Aug 02 09:56:00 +0530 2018 by leftnoteasy from branch-3.1.1
Cluster ID:	CID-621a8db9-02fd-4cd2-b100-58f18157f3cb
Block Pool ID:	BP-1751532922-127.0.1.1-1749532332875

Below the table is a "Summary" section. It states: "Security is off.", "Safemode is off.", "1 files and directories, 0 blocks (0 replicated blocks, 0 erasure coded block groups) = 1 total filesystem object(s).", "Heap Memory used 72.54 MB of 235.5 MB Heap Memory. Max Heap Memory is 860.5 MB.", and "Non Heap Memory used 46.6 MB of 48 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.". At the bottom, a table shows "Configured Capacity: 38.58 GB".

<http://localhost:8088>

The screenshot shows the Hadoop JobTracker Web Interface. The browser address bar displays `localhost:8088/cluster`. The page features the Hadoop logo and the title "All Applications". On the left is a sidebar with a "Cluster" section containing links: About, Nodes, Node Labels, Applications, NEW, NEW SAVING, SUBMITTED, ACCEPTED, RUNNING, FINISHED, FAILED, KILLED, and Scheduler. Below the sidebar is a "Tools" button. The main content area displays "Cluster Metrics" and "Cluster Nodes Metrics".

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Memory Used	Memory Total	Memory Free
0	0	0	0	0	0 B	8 GB	0 B

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes	Unhealthy Nodes
1	0	0	0	0

Below the node metrics is the "Scheduler Metrics" section, which includes a table with columns: Scheduler Type, Scheduling Resource Type, Minimum Allocation, and Maximum Allocation. The values are: Capacity Scheduler, [memory-mb (unit=M), vcores], <memory:1024, vCores:1>, and <memory:8192, vCores:1> respectively.

At the bottom, there is a table for "All Applications" with columns: ID, User, Name, Application Type, Queue, Application Priority, StartTime, FinishTime, State, FinalStatus, Running Containers, Allocated CPU Vcores, Allocated Memory MB, and Reserved Memory MB. The table is currently empty, showing "No data available in table" and "Showing 0 to 0 of 0 entries".

Practical No - 2

Aim: Implementing Map-Reduce Program for Word Count problem,

```
WC_Mapper.java package  
com.wordcountproblem;
```

```
import java.io.IOException;  
import java.util.StringTokenizer;
```

```
import org.apache.hadoop.io.IntWritable; import  
org.apache.hadoop.io.LongWritable; import  
org.apache.hadoop.io.Text; import  
org.apache.hadoop.mapred.MapReduceBase; import  
org.apache.hadoop.mapred.Mapper; import  
org.apache.hadoop.mapred.OutputCollector; import  
org.apache.hadoop.mapred.Reporter;
```

```
/**
```

```
* WC_Mapper is the Mapper class for the Word Count problem.  
* It extends MapReduceBase and implements the Mapper interface.
```

```
*
```

```
* The map method takes a line of text as input, tokenizes it into words, * and emits each word along  
with a count of 1.
```

```
*/
```

```
public class WC_Mapper extends MapReduceBase implements  
Mapper<LongWritable, Text, Text, IntWritable> {
```

```
    // A constant IntWritable with value 1, used for counting each word occurrence.  
    private final static IntWritable one = new IntWritable(1);    // A Text object to  
    hold the current word being processed.    private Text word = new Text();
```

```
    /**
```

```
* The map method processes a single line of input text.
```

```
*
```

```
* @param key The byte offset of the current line in the input file (not used in this mapper).
```

```
* @param value The entire line of text as a Text object.
```

```
* @param output An OutputCollector to emit key-value pairs (word, 1).
```

```
* @param reporter A Reporter object for reporting progress and counters (not used in this mapper).
```

```
* @throws IOException If an I/O error occurs.
```

```
*/
```

```
    public void map(LongWritable key, Text value,  
                    OutputCollector<Text, IntWritable> output,
```

```

        Reporter reporter) throws IOException {

    // Convert the input Text value to a Java String.
    String line = value.toString();

    // Create a StringTokenizer to break the line into words.
    // By default, StringTokenizer uses whitespace as delimiters.
    StringTokenizer tokenizer = new StringTokenizer(line);

    // Iterate through each token (word) in the line.
    while (tokenizer.hasMoreTokens()) {
        // Set the 'word' Text object to the current token.
        word.set(tokenizer.nextToken());
        // Emit the word and a count of 1 to the OutputCollector.
        output.collect(word, one);
    }
}
}

```

WC_Reducer.java package
com.wordcountproblem;

import java.io.IOException;
import java.util.Iterator;

import org.apache.hadoop.io.IntWritable; import
org.apache.hadoop.io.Text; import
org.apache.hadoop.mapred.MapReduceBase; import
org.apache.hadoop.mapred.OutputCollector; import
org.apache.hadoop.mapred.Reducer; import
org.apache.hadoop.mapred.Reporter;

/**

* WC_Reducer is the Reducer class for the Word Count problem.

* It extends MapReduceBase and implements the Reducer interface.

*

* The reduce method takes a word as a key and an iterator of counts (1s) * as values. It sums these counts to get the total frequency of the word * and emits the word along with its total count.

*/

public class WC_Reducer extends MapReduceBase implements
Reducer<Text, IntWritable, Text, IntWritable> {

/**

* The reduce method aggregates the counts for a given word.

```

*
* @param key The word (Text) whose counts are to be summed.
* @param values An Iterator of IntWritable objects, each representing a count of 1 * for the
given word from the mappers.
* @param output An OutputCollector to emit the final word and its total count.
* @param reporter A Reporter object for reporting progress and counters (not used in this reducer).
* @throws IOException If an I/O error occurs.
*/
public void reduce(Text key, Iterator<IntWritable> values,
                  OutputCollector<Text, IntWritable> output,
Reporter reporter) throws IOException {

    // Initialize a sum variable to accumulate the counts for the current word.
    int sum = 0;

    // Iterate through all the IntWritable values (which are all '1's from the mappers)
    // associated with the current word (key). while (values.hasNext()) {
        // Get the integer value from the current IntWritable and add it to the sum.
    sum += values.next().get();
    }

    // After summing all the counts for the word, emit the word (key)
    // and its total count (sum) as a new IntWritable object.
    output.collect(key, new IntWritable(sum));
}
}

```

```

WC_Runner.java package com.wordcountproblem;
import java.io.IOException; import
org.apache.hadoop.fs.Path; import
org.apache.hadoop.io.IntWritable; import
org.apache.hadoop.io.Text; import
org.apache.hadoop.mapred.FileInputFormat; import
org.apache.hadoop.mapred.FileOutputFormat; import
org.apache.hadoop.mapred.JobClient; import
org.apache.hadoop.mapred.JobConf; import
org.apache.hadoop.mapred.TextInputFormat; import
org.apache.hadoop.mapred.TextOutputFormat; public
class WC_Runner {
public static void main(String[] args) throws IOException {
JobConf conf = new JobConf(WC_Runner.class);
conf.setJobName("WordCount");
conf.setOutputKeyClass(Text.class);

```

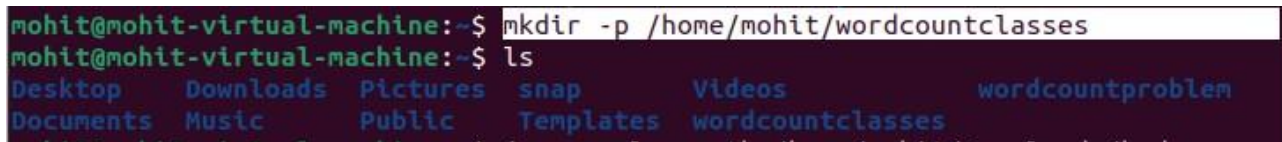
```

conf.setOutputValueClass(IntWritable.class);
conf.setMapperClass(WC_Mapper.class);
conf.setCombinerClass(WC_Reducer.class);
conf.setReducerClass(WC_Reducer.class);
conf.setInputFormat(TextInputFormat.class);
conf.setOutputFormat(TextOutputFormat.class);
FileInputFormat.setInputPaths(conf,new Path(args[0]));
FileOutputFormat.setOutputPath(conf,new Path(args[1]));
JobClient.runJob(conf);
}
}

```

Compiling and making jar files (make sure to login as root user)

```
mkdir -p /home/mohit/wordcountclasses
```



```

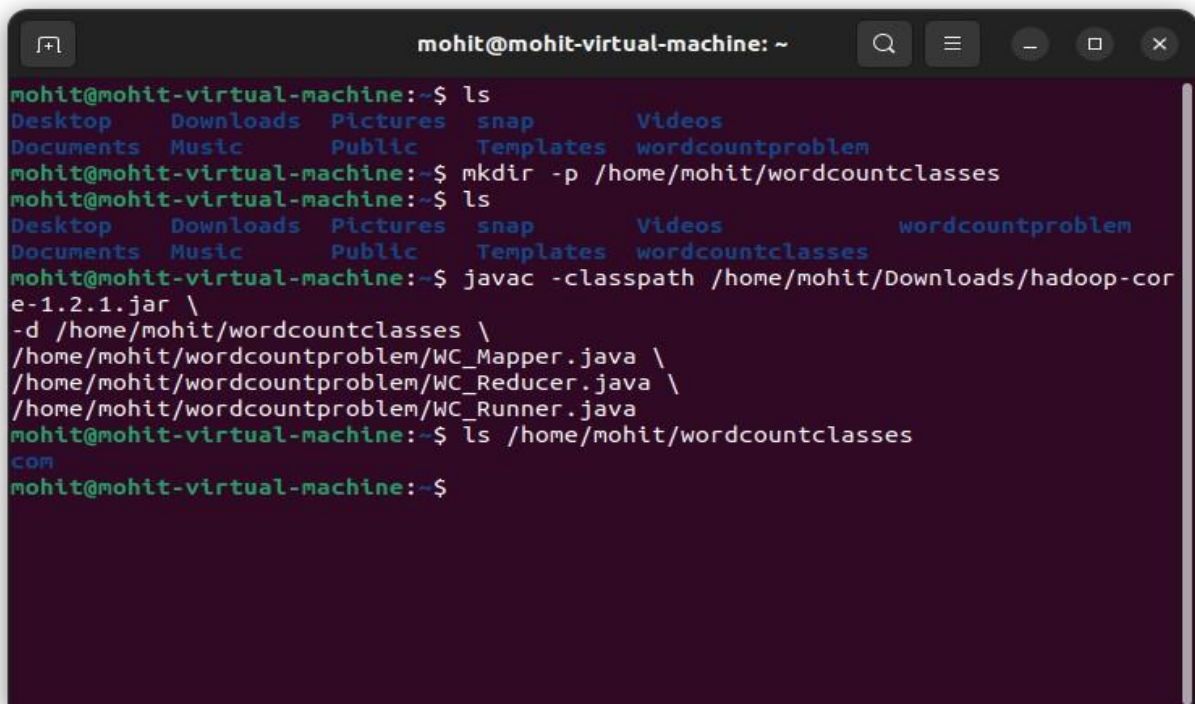
mohit@mohit-virtual-machine:~$ mkdir -p /home/mohit/wordcountclasses
mohit@mohit-virtual-machine:~$ ls
Desktop  Downloads  Pictures  snap      Videos  wordcountproblem
Documents Music      Public   Templates wordcountclasses

```

```

javac -classpath /home/mohit/Downloads/hadoop-core-1.2.1.jar \
-d /home/mohit/wordcountclasses \
/home/mohit/wordcountproblem/WC_Mapper.java \
/home/mohit/wordcountproblem/WC_Reducer.java \
/home/mohit/wordcountproblem/WC_Runner.java

```



```

mohit@mohit-virtual-machine: ~
mohit@mohit-virtual-machine:~$ ls
Desktop  Downloads  Pictures  snap      Videos  wordcountproblem
Documents Music      Public   Templates wordcountclasses
mohit@mohit-virtual-machine:~$ mkdir -p /home/mohit/wordcountclasses
mohit@mohit-virtual-machine:~$ ls
Desktop  Downloads  Pictures  snap      Videos  wordcountclasses  wordcountproblem
Documents Music      Public   Templates
mohit@mohit-virtual-machine:~$ javac -classpath /home/mohit/Downloads/hadoop-core-1.2.1.jar \
-d /home/mohit/wordcountclasses \
/home/mohit/wordcountproblem/WC_Mapper.java \
/home/mohit/wordcountproblem/WC_Reducer.java \
/home/mohit/wordcountproblem/WC_Runner.java
mohit@mohit-virtual-machine:~$ ls /home/mohit/wordcountclasses
COM
mohit@mohit-virtual-machine:~$

```



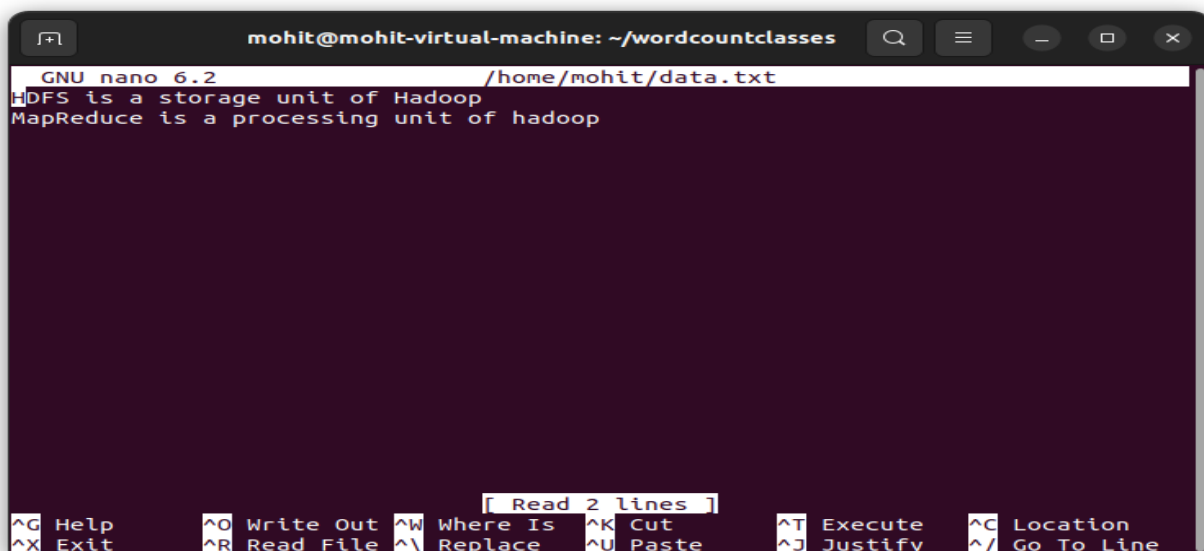
```
cd /home/mohit/wordcountclasses jar -cvf
```

```
/home/mohit/wordcountproblem.jar com/
```

```
mohit@mohit-virtual-machine:~$ cd /home/mohit/wordcountclasses
jar -cvf /home/mohit/wordcountproblem.jar com/
added manifest
adding: com/(in = 0) (out= 0)(stored 0%)
adding: com/wordcountproblem/(in = 0) (out= 0)(stored 0%)
adding: com/wordcountproblem/WC_Runner.class(in = 1503) (out= 730)(deflated 51%)
adding: com/wordcountproblem/WC_Mapper.class(in = 1883) (out= 777)(deflated 58%)
adding: com/wordcountproblem/WC_Reducer.class(in = 1552) (out= 621)(deflated 59%)
)
```

```
nano /home/mohit/data.txt
```

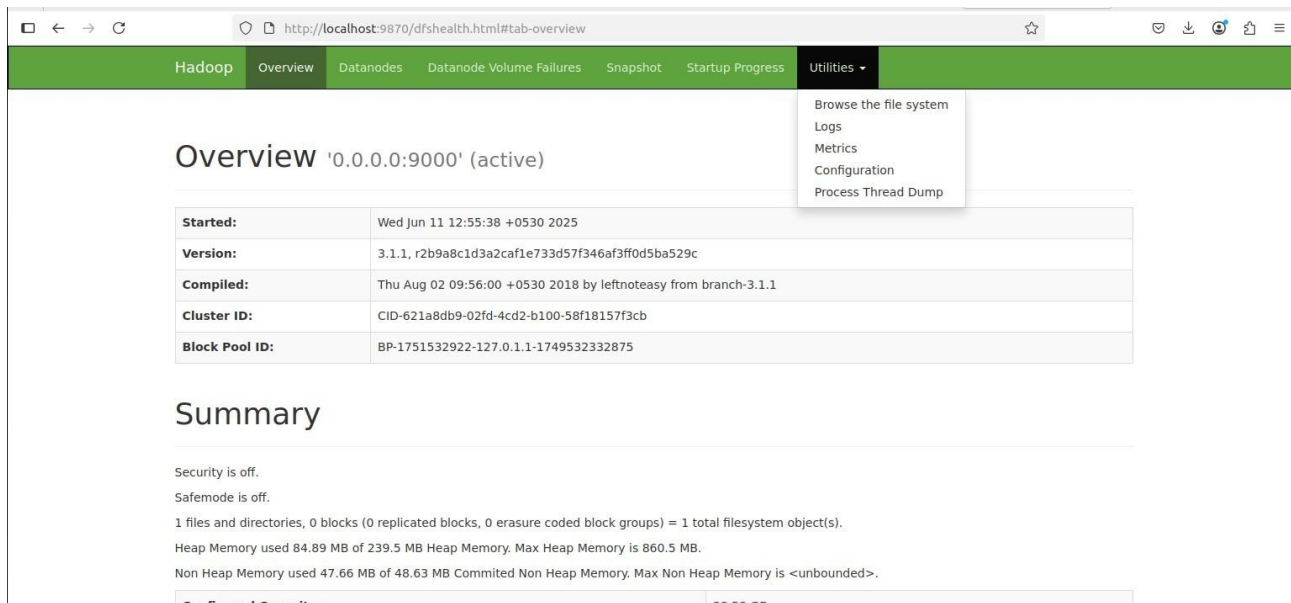
```
mohit@mohit-virtual-machine:~/wordcountclasses$ nano /home/mohit/data.txt
```



Start Hadoop server and yarn server (start-dfs.sh and start-yarn.sh)

```
mohit@mohit-virtual-machine:~$ 
mohit@mohit-virtual-machine:~$ sudo su - hadoop
hadoop@mohit-virtual-machine:~$ start-dfs.sh
Starting namenodes on [0.0.0.0]
Starting datanodes
Starting secondary namenodes [mohit-virtual-machine]
hadoop@mohit-virtual-machine:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
hadoop@mohit-virtual-machine:~$
```

Create a folder in Hadoop and upload the data.txt file



The screenshot shows the Hadoop Overview page in a web browser. The browser address bar displays `http://localhost:9870/dfshealth.html#tab-overview`. The page has a green navigation bar with tabs: Hadoop, Overview, Datanodes, Datanode Volume Failures, Snapshot, Startup Progress, and Utilities. The Utilities dropdown menu is open, showing options: Browse the file system, Logs, Metrics, Configuration, and Process Thread Dump.

Overview '0.0.0.0:9000' (active)

Started:	Wed Jun 11 12:55:38 +0530 2025
Version:	3.1.1, r2b9a8c1d3a2caf1e733d57f346af3ff0d5ba529c
Compiled:	Thu Aug 02 09:56:00 +0530 2018 by leftnoteasy from branch-3.1.1
Cluster ID:	CID-621a8db9-02fd-4cd2-b100-58f18157f3cb
Block Pool ID:	BP-1751532922-127.0.1.1-1749532332875

Summary

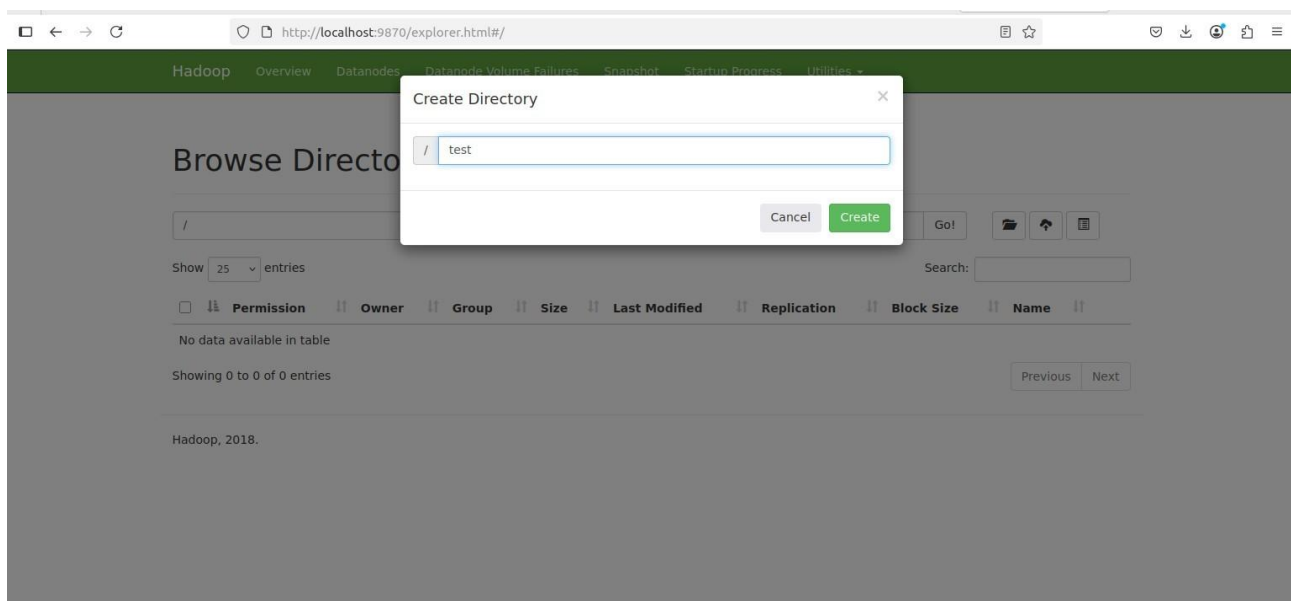
Security is off.
Safemode is off.

1 files and directories, 0 blocks (0 replicated blocks, 0 erasure coded block groups) = 1 total filesystem object(s).

Heap Memory used 84.89 MB of 239.5 MB Heap Memory. Max Heap Memory is 860.5 MB.

Non Heap Memory used 47.66 MB of 48.63 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity	38.63 GB
---------------------	----------



The screenshot shows the Hadoop Browse Directory page. The browser address bar displays `http://localhost:9870/explorer.html/#/`. The page has a green navigation bar with tabs: Hadoop, Overview, Datanodes, Datanode Volume Failures, Snapshot, Startup Progress, and Utilities. A "Create Directory" dialog box is open, showing a text input field with the value "test" and buttons for "Cancel" and "Create".

Browse Directory

/ test

Cancel Create

Go! [Icons]

Show 25 entries

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
No data available in table							

Showing 0 to 0 of 0 entries

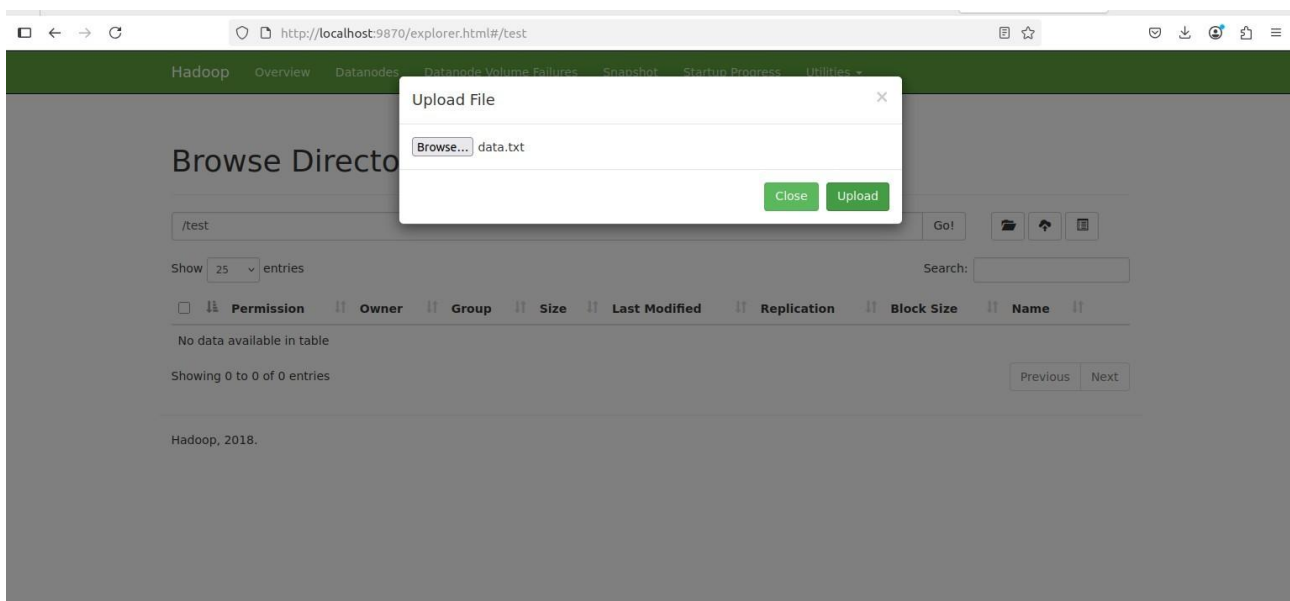
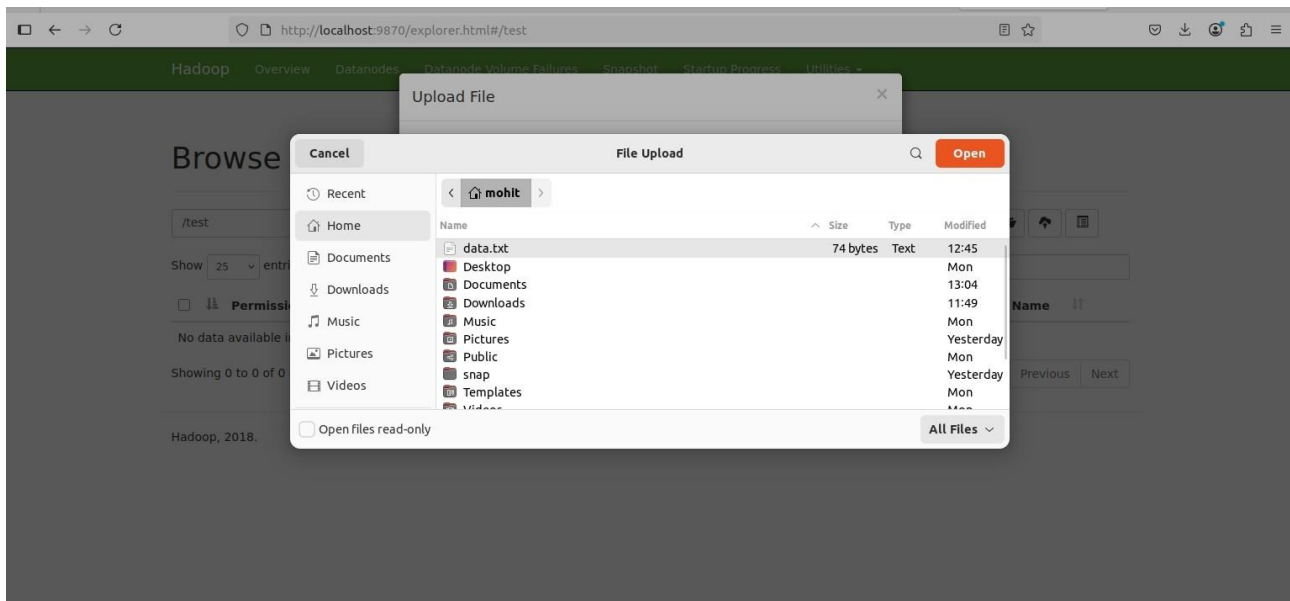
Previous Next

Hadoop, 2018.

The screenshot shows the Hadoop Explorer web interface at `http://localhost:9870/explorer.html#/`. The navigation bar includes links for Hadoop, Overview, Datanodes, Datanode Volume Failures, Snapshot, Startup Progress, and Utilities. The main heading is "Browse Directory". Below it is a search bar with a "Go!" button and icons for file operations. A table lists the contents of the root directory, showing a single entry named "test" with permissions "drwxr-xr-x", owner "dr.who", group "supergroup", size "0 B", and last modified date "Jun 11 13:03". The table has columns for Permission, Owner, Group, Size, Last Modified, Replication, Block Size, and Name. At the bottom, it says "Showing 1 to 1 of 1 entries" and "Hadoop, 2018."

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	dr.who	supergroup	0 B	Jun 11 13:03	0	0 B	test

The screenshot shows the Hadoop Explorer web interface at `http://localhost:9870/explorer.html#/test`. The "test" directory is selected in the breadcrumb. An "Upload File" dialog box is open in the center, showing a "Browse..." button and the message "No files selected." Below this, there is a "No files selected." message and two buttons: "Close" and "Upload". The background interface shows the "test" directory is empty, with "Showing 0 to 0 of 0 entries" and "No data available in table".



Compiling (make sure to login as Hadoop user)

Use cd command to go to directory where wordcountproblem.jar is saved

`hadoop jar ~/wordcountproblem.jar com.wordcountproblem.WC_Runner /test/data.txt /r_output`

```
hadoop@mohit-virtual-machine:~$ ls
hadoop-3.1.1.tar.gz  hdfs  snap  wordcountproblem.jar
hadoop@mohit-virtual-machine:~$
hadoop@mohit-virtual-machine:~$
hadoop@mohit-virtual-machine:~$ hadoop jar ~/wordcountproblem.jar com.wordcountp
roblem.WC_Runner /test/data.txt /r_output
2025-06-11 14:14:55,042 INFO client.RMProxy: Connecting to ResourceManager at /0
.0.0.0:8032
```

Running

hdfs dfs -cat /r_output/part-00000

```
hadoop@mohit-virtual-machine:~$  
hadoop@mohit-virtual-machine:~$ hdfs dfs -cat /r_output/part-00000  
HDFS      1  
Hadoop    1  
MapReduce      1  
a          2  
hadoop      1  
is          2  
of          2  
processing    1  
storage      1  
unit         2  
hadoop@mohit-virtual-machine:~$
```


Practical No – 3

Aim: Write a program to Implement a tri-gram model

```
from collections import defaultdict
import random
```

```
def generate_trigrams(text):
    """
    Builds a trigram model from input text.
    A trigram is a sequence of three consecutive words.
    """
    words = text.split()
    trigram_model = defaultdict(list)
    for i in range(len(words) - 2):
        key = (words[i], words[i + 1])
        next_word = words[i + 2]
        trigram_model[key].append(next_word)
    return trigram_model
```

```
def generate_text(model, start_words, num_words=10):
    """
    Generate new text using a trigram model starting from a given word pair.
    """
    if start_words not in model:
        return "The given start words are not in the model."

    current_words = list(start_words)
    output = list(current_words)

    for _ in range(num_words):
        next_words = model.get(tuple(current_words))
        if not next_words:
            break
        next_word = random.choice(next_words)
        output.append(next_word)
        current_words = [current_words[1], next_word]

    return ''.join(output)
```

```
input_text = "I wish I may I wish I might"
```

```
model = generate_trigrams(input_text)
print ( "Tri-gram Model:" )
for key, values in model.items():
```

```
print (f" {key} -> {values}" )
```

```
print ( "\nGenerated Text:" )
```

```
start = ('I', 'wish' )
```

```
print(generate_text(model, start, num_words=10))
```



Tri-gram Model:

('I', 'wish') -> ['I', 'I']

('wish', 'I') -> ['may', 'might']

('I', 'may') -> ['I']

('may', 'I') -> ['wish']

Generated Text:

I wish I might

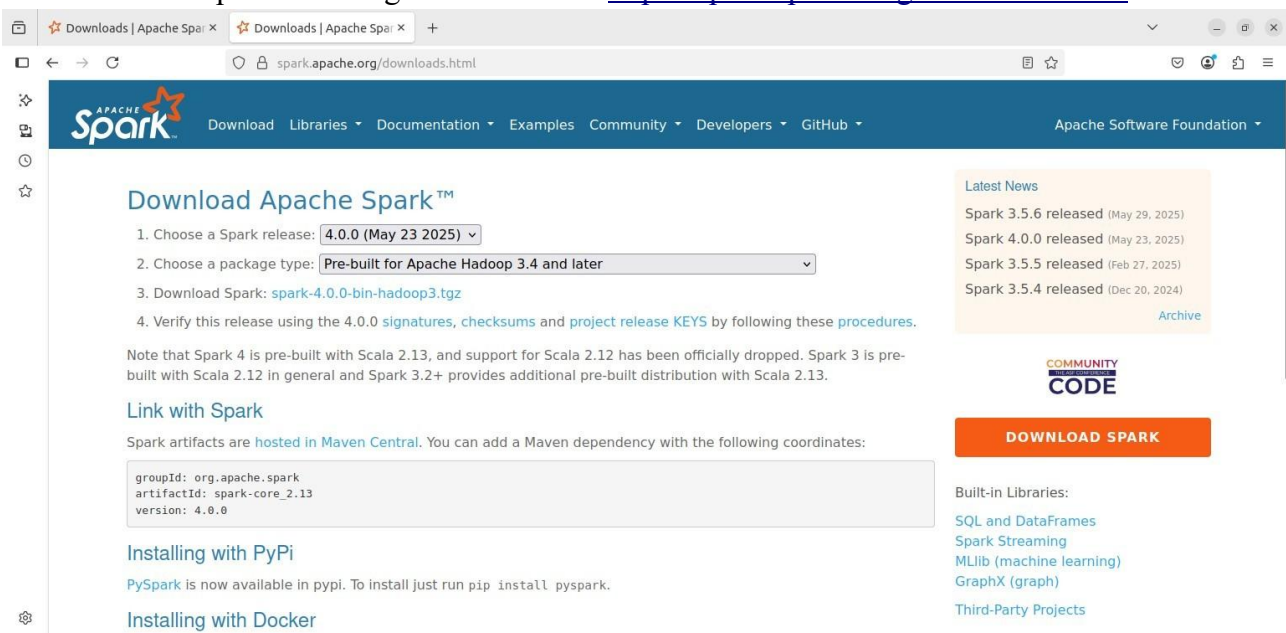
Practical No – 4

Aim: Download and install Spark. Create Graphical data and access the graphical data using Spark

Introduction:

Apache Spark is a powerful, open-source, distributed computing system used for big data processing and analytics. It's designed to handle both batch and real-time workloads, making it suitable for a wide range of applications, including machine learning, interactive queries, and stream processing. Spark is known for its speed and ease of use, providing a user-friendly interface for managing and executing distributed data processing tasks

for intalltion of spart visit the given link below <https://spark.apache.org/downloads.html>



STEP 1: Download & Install Apache Spark

Open terminal and run: now check your java

version

```
mohit@mohit-virtual-machine:~$ java -version
openjdk version "17.0.15" 2025-04-15
OpenJDK Runtime Environment (build 17.0.15+6-Ubuntu-0ubuntu122.04)
OpenJDK 64-Bit Server VM (build 17.0.15+6-Ubuntu-0ubuntu122.04, mixed mode, sharing)
mohit@mohit-virtual-machine:~$
```

1. Download Spark (with Hadoop 3)

wget https://downloads.apache.org/spark/spark-4.0.0/spark-4.0.0-bin-hadoop3.tgz

2. Extract the archive tar -xzf

spark-4.0.0-bin-hadoop3.tgz

3. Move to a simpler location mv
spark-4.0.0-bin-hadoop3 ~/spark

```
Documents Pictures Templates wordcountproblem
mohit@mohit-virtual-machine:~$ cd Downloads
mohit@mohit-virtual-machine:~/Downloads$ ls
hadoop-core-1.2.1.jar 'Practical no-1.pdf' spark-4.0.0-bin-hadoop3.tgz
mohit@mohit-virtual-machine:~/Downloads$ tar -xzf spark-4.0.0-bin-hadoop3.tgz
mohit@mohit-virtual-machine:~/Downloads$ mv spark-4.0.0-bin-hadoop3 ~/spark
mohit@mohit-virtual-machine:~/Downloads$
```

STEP 2: Set Environment Variables (Optional but recommended)

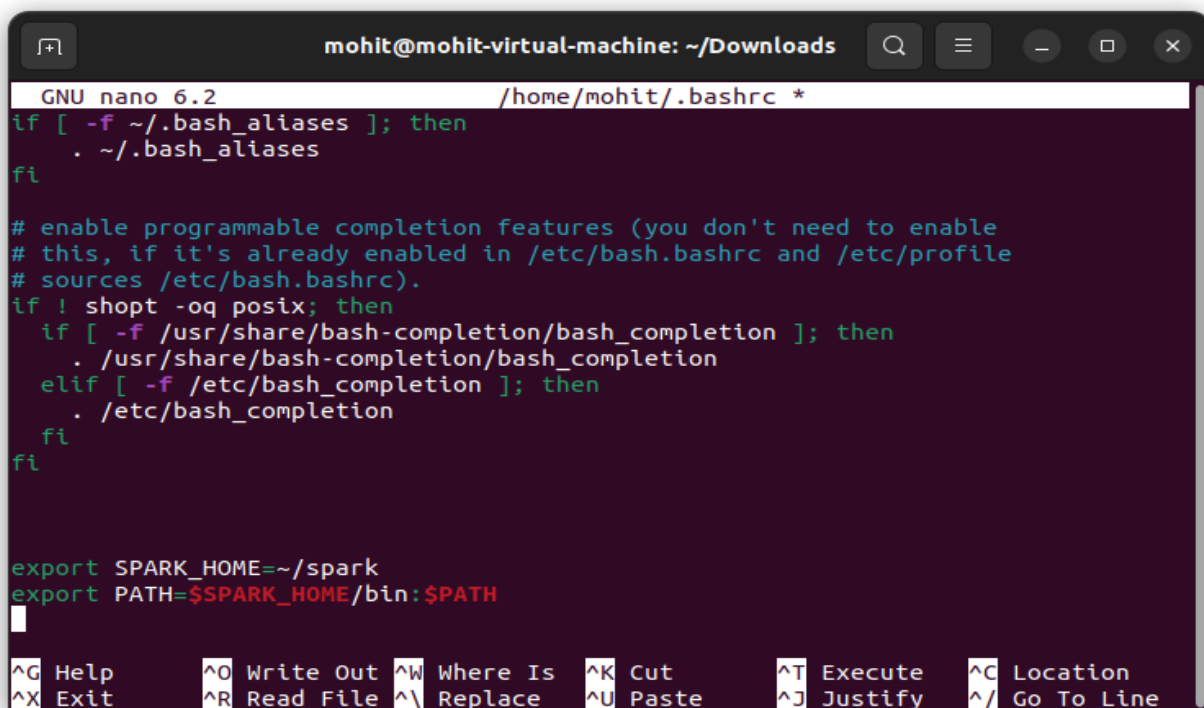
Edit your ~/.bashrc or ~/.zshrc:

nano ~/.bashrc

Add at the bottom:

export SPARK_HOME=~/spark export

PATH=\$SPARK_HOME/bin:\$PATH



```
mohit@mohit-virtual-machine: ~/Downloads
GNU nano 6.2 /home/mohit/.bashrc *
if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
    if [ -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi

export SPARK_HOME=~/spark
export PATH=$SPARK_HOME/bin:$PATH
```

Then apply it:

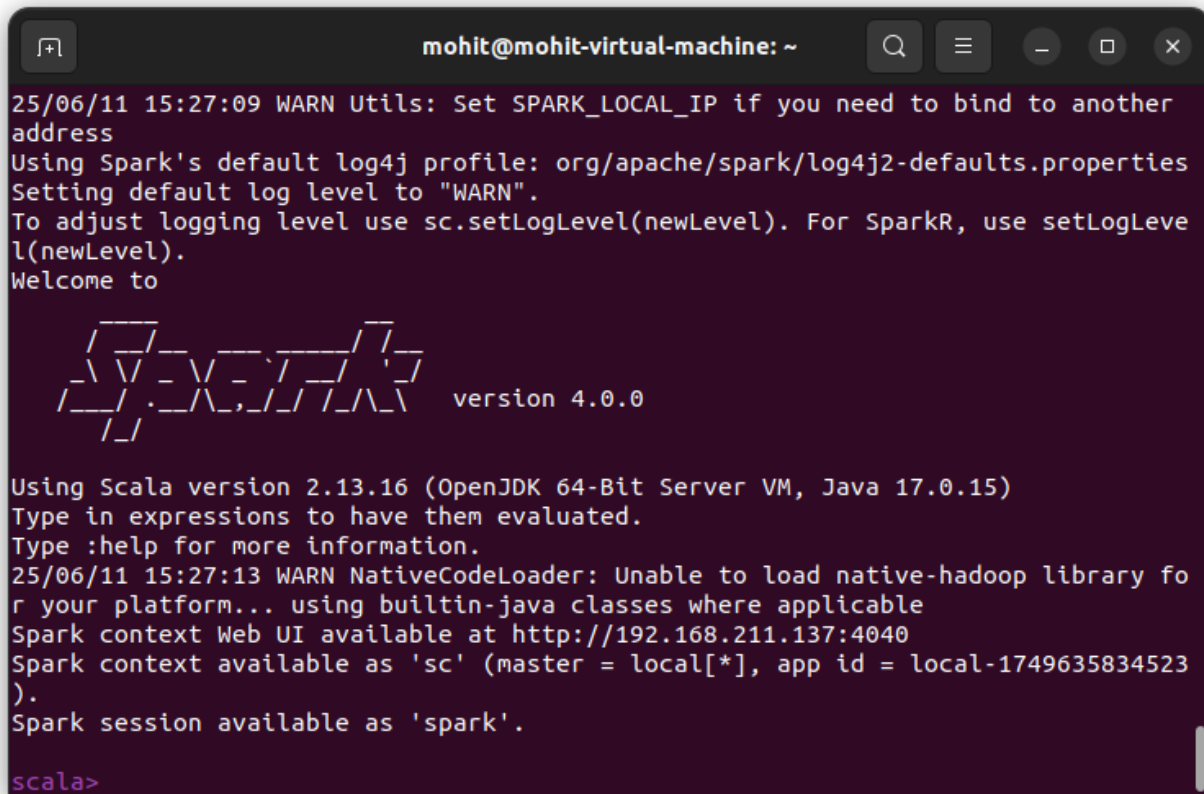
source ~/.bashrc

```

mohit@mohit-virtual-machine:~/Downloads$ source ~/.bashrc
mohit@mohit-virtual-machine:~/Downloads$

```

STEP 3: Run spark-shell (Scala-based REPL for Spark) spark-shell



```

25/06/11 15:27:09 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(
newLevel).
Welcome to

  ____      __
 / ___/____/ /  ___
/ /  / __/ _ \/ _ \
/ ___/ __/ ___// ___/
/_/   \___/_/ \___/

version 4.0.0

Using Scala version 2.13.16 (OpenJDK 64-Bit Server VM, Java 17.0.15)
Type in expressions to have them evaluated.
Type :help for more information.
25/06/11 15:27:13 WARN NativeCodeLoader: Unable to load native-hadoop library fo
r your platform... using builtin-java classes where applicable
Spark context Web UI available at http://192.168.211.137:4040
Spark context available as 'sc' (master = local[*], app id = local-1749635834523
).
Spark session available as 'spark'.

scala>

```

Create Graphical data and access the graphical data using Spark

Type the code line by line in the spark-shell

Type spark-shell to access the shell

Entering graphical data in spark shell

#creating graphical data in graphx

import org.apache.spark.graphx._

creating own data type case class

User(name: String, age: Int) val users

= List((1L, User("Alex", 26)), (2L,

User("Bill", 42)), (3L, User("Carol",

18)), (4L, User("Dave",16)), (5L,

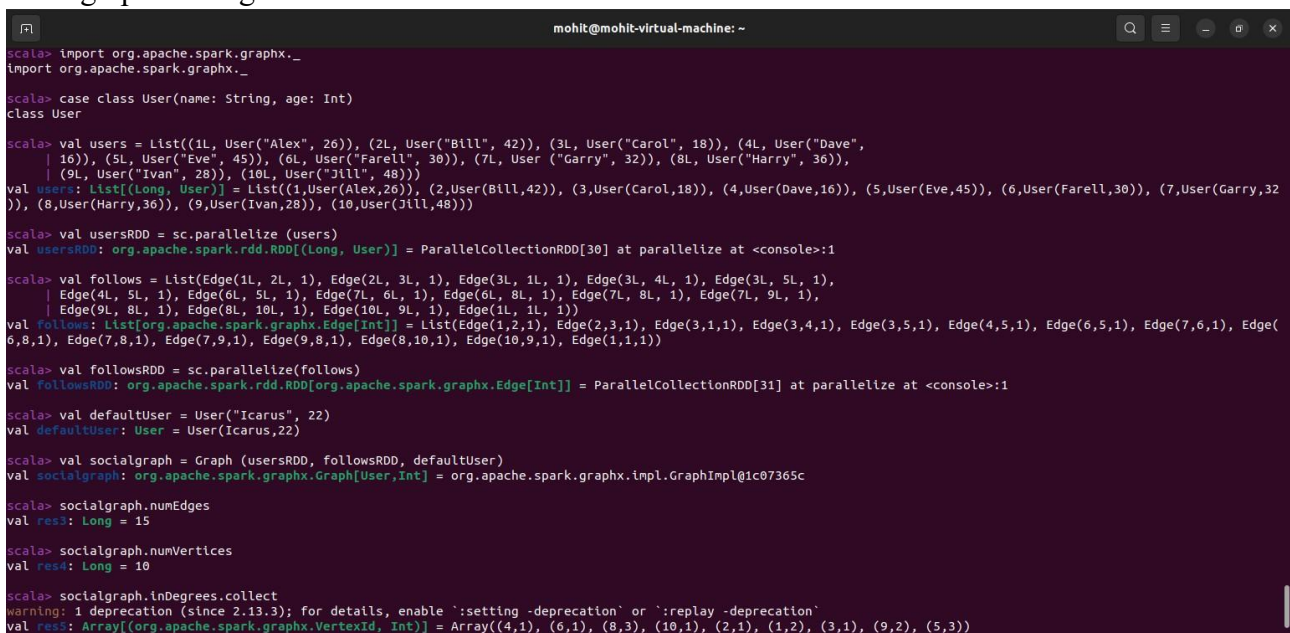

```
User("Eve", 45)), (6L, User("Farell",
30)), (7L, User ("Garry", 32)), (8L,
User("Harry", 36)), (9L, User("Ivan", 28)), (10L, User("Jill", 48)))
```

```
val usersRDD = sc.parallelize (users)
val follows = List(Edge(1L, 2L, 1), Edge(2L, 3L, 1), Edge(3L, 1L, 1), Edge(3L, 4L, 1), Edge(3L,
5L, 1),
Edge(4L, 5L, 1), Edge(6L, 5L, 1), Edge(7L, 6L, 1), Edge(6L, 8L, 1), Edge(7L, 8L, 1), Edge(7L, 9L,
1), Edge(9L, 8L, 1), Edge(8L, 10L, 1), Edge(10L, 9L, 1), Edge(1L, 1L, 1))
```

```
val followsRDD = sc.parallelize(follows)
```

```
# creating user to access data 19 val
defaultUser = User("Icarus", 22)
val socialgraph = Graph (usersRDD, followsRDD, defaultUser)
```

```
#Access data of the graph
socialgraph.numEdges
socialgraph.numVertices
socialgraph.inDegrees.collect
socialgraph.outDegrees.collect
```



```
mohit@mohit-virtual-machine: ~
scala> import org.apache.spark.graphx._
import org.apache.spark.graphx._

scala> case class User(name: String, age: Int)
class User

scala> val users = List((1L, User("Alex", 26)), (2L, User("Bill", 42)), (3L, User("Carol", 18)), (4L, User("Dave",
16)), (5L, User("Eve", 45)), (6L, User("Farell", 30)), (7L, User ("Garry", 32)), (8L, User("Harry", 36)),
(9L, User("Ivan", 28)), (10L, User("Jill", 48)))
val users: List[(Long, User)] = List((1,User(Alex,26)), (2,User(Bill,42)), (3,User(Carol,18)), (4,User(Dave,16)), (5,User(Eve,45)), (6,User(Farell,30)), (7,User(Garry,32)), (8,User(Harry,36)), (9,User(Ivan,28)), (10,User(Jill,48)))

scala> val usersRDD = sc.parallelize (users)
val usersRDD: org.apache.spark.rdd.RDD[(Long, User)] = ParallelCollectionRDD[30] at parallelize at <console>:1

scala> val follows = List(Edge(1L, 2L, 1), Edge(2L, 3L, 1), Edge(3L, 1L, 1), Edge(3L, 4L, 1), Edge(3L, 5L, 1),
Edge(4L, 5L, 1), Edge(6L, 5L, 1), Edge(7L, 6L, 1), Edge(6L, 8L, 1), Edge(7L, 8L, 1), Edge(7L, 9L, 1),
Edge(9L, 8L, 1), Edge(8L, 10L, 1), Edge(10L, 9L, 1), Edge(1L, 1L, 1))
val follows: List[org.apache.spark.graphx.Edge[Int]] = List(Edge(1,2,1), Edge(2,3,1), Edge(3,1,1), Edge(3,4,1), Edge(3,5,1), Edge(4,5,1), Edge(6,5,1), Edge(7,6,1), Edge(6,8,1), Edge(7,8,1), Edge(7,9,1), Edge(9,8,1), Edge(8,10,1), Edge(10,9,1), Edge(1,1,1))

scala> val followsRDD = sc.parallelize(follows)
val followsRDD: org.apache.spark.rdd.RDD[org.apache.spark.graphx.Edge[Int]] = ParallelCollectionRDD[31] at parallelize at <console>:1

scala> val defaultUser = User("Icarus", 22)
val defaultUser: User = User(Icarus,22)

scala> val socialgraph = Graph (usersRDD, followsRDD, defaultUser)
val socialgraph: org.apache.spark.graphx.Graph[User,Int] = org.apache.spark.graphx.impl.GraphImpl@1c07365c

scala> socialgraph.numEdges
val res3: Long = 15

scala> socialgraph.numVertices
val res4: Long = 10

scala> socialgraph.inDegrees.collect
warning: 1 deprecation (since 2.13.3): for details, enable `:setting -deprecation` or `:replay -deprecation`
val res5: Array[(org.apache.spark.graphx.VertexId, Int)] = Array((4,1), (6,1), (8,3), (10,1), (2,1), (1,2), (3,1), (9,2), (5,3))
```

press ctrl + D for exit the shell

Practical No - 5

Aim: Write a Spark code for the given application and handle error and recovery of data

Introduction:

Scala provides robust constructs for functional error handling. These include:

- Try, Success, Failure – for wrapping computations that may fail.
- Option, Some, None – for handling optional values.
- Either, Left, Right – for results that can be two possible types (usually success or failure).

Install sbt (Scala Build Tool)

```
sudo apt update
```

```
install curl -y
```

```
mohit@mohit-virtual-machine:~$ sudo apt install curl -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  curl
```

```
sudo apt install gnupg -y
```

```
mohit@mohit-virtual-machine:~$ sudo apt update
sudo apt install gnupg -y
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
```

```
sudo apt-key adv --keyserver hkp://keyserver.ubuntu.com:80 \
--recv-keys 2EE0EA64E40A89B84B2DF73499E82A75642AC823
```

```
mohit@mohit-virtual-machine:~$ sudo apt-key adv --keyserver hkp://keyserver.ubuntu.com:80 \
--recv-keys 2EE0EA64E40A89B84B2DF73499E82A75642AC823
Warning: apt-key is deprecated. Manage keyring files in trusted.gpg.d instead (see apt-key(8)).
Executing: /tmp/apt-key-gpghome.RQloTywQC2/gpg.1.sh --keyserver hkp://keyserver.
```

```
echo "deb https://repo.scala-sbt.org/scalasbt/debian all main" | \
sudo tee /etc/apt/sources.list.d/sbt.list
```

```
echo "deb https://repo.scala-sbt.org/scalasbt/debian /" | \
sudo tee /etc/apt/sources.list.d/sbt_old.list
```

```
mohit@mohit-virtual-machine:~$ echo "deb https://repo.scala-sbt.org/scalasbt/debian all main" | \
  sudo tee /etc/apt/sources.list.d/sbt.list
echo "deb https://repo.scala-sbt.org/scalasbt/debian /" | \
  sudo tee /etc/apt/sources.list.d/sbt_old.list
deb https://repo.scala-sbt.org/scalasbt/debian all main
deb https://repo.scala-sbt.org/scalasbt/debian /
```

```
sudo apt update sudo
```

```
apt install sbt -y
```

```
mohit@mohit-virtual-machine:~$ sudo apt update
sudo apt install sbt -y
Hit:1 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:5 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
```

```
echo "deb [signed-by=/usr/share/keyrings/sbt-archive-keyring.gpg]
```

```
https://repo.scala-sbt.org/scalasbt/debian all main" | sudo tee /etc/apt/sources.list.d/sbt.list
```

```
mohit@mohit-virtual-machine:~$ echo "deb [signed-by=/usr/share/keyrings/sbt-archive-keyring.gpg] https://repo.scala-sbt.org/scalasbt/debian all main" | sudo tee /etc/apt/sources.list.d/sbt.list
deb [signed-by=/usr/share/keyrings/sbt-archive-keyring.gpg] https://repo.scala-sbt.org/scalasbt/debian all main
```

```
curl -sL "https://keyserver.ubuntu.com/pks/lookup?
```

```
op=get&search=0x2EE0EA64E40A89B84B2DF73499E82A75642AC823" | sudo apt-key add -
```

```
mohit@mohit-virtual-machine:~$ curl -sL "https://keyserver.ubuntu.com/pks/lookup?op=get&search=0x2EE0EA64E40A89B84B2DF73499E82A75642AC823" | sudo apt-key add -
Warning: apt-key is deprecated. Manage keyring files in trusted.gpg.d instead (see apt-key(8)).
OK
```

```
sudo apt-get update
```

Verify sbt installation

```
Sbt
```

```

mohit@mohit-virtual-machine:~$ sbt
[error] Neither build.sbt nor a 'project' directory in the current directory: /home/mohit
[error] run 'sbt new', touch build.sbt, or run 'sbt --allow-empty'.
[error]
[error] To opt out of this check, create /home/mohit/.config/sbt/sbtopts with:
[error] --allow-empty
mohit@mohit-virtual-machine:~$

```

Install scala

wget <https://downloads.lightbend.com/scala/2.12.18/scala-2.12.18.deb>

sudo dpkg -i scala-2.12.18.deb

```

mohit@mohit-virtual-machine:~$ scala -version
Scala code runner version 2.12.18 -- Copyright 2002-2023, LAMP/EPFL and Lightbend, Inc.

```

Create Scala source file nano

ExceptionHandlingTest.scala

```

mohit@mohit-virtual-machine:~$ nano ExceptionHandlingTest.scala
mohit@mohit-virtual-machine:~$

```

```

import org.apache.spark.sql.SparkSession

object ExceptionHandlingTest {
  def
  main(args: Array[String]): Unit = {
    val
    spark = SparkSession
      .builder()
      .appName("ExceptionHandlingTest")
      .master("local[*]") // Optional: helpful for local debugging
      .getOrCreate()

    val rdd = spark.sparkContext.parallelize(0 until spark.sparkContext.defaultParallelism)

    rdd.foreach { i =>
    try {
      if (math.random > 0.75) {
        throw new Exception("Testing exception handling")
      } else {
        println(s"Task $i completed successfully")
      }
    }
  }

```

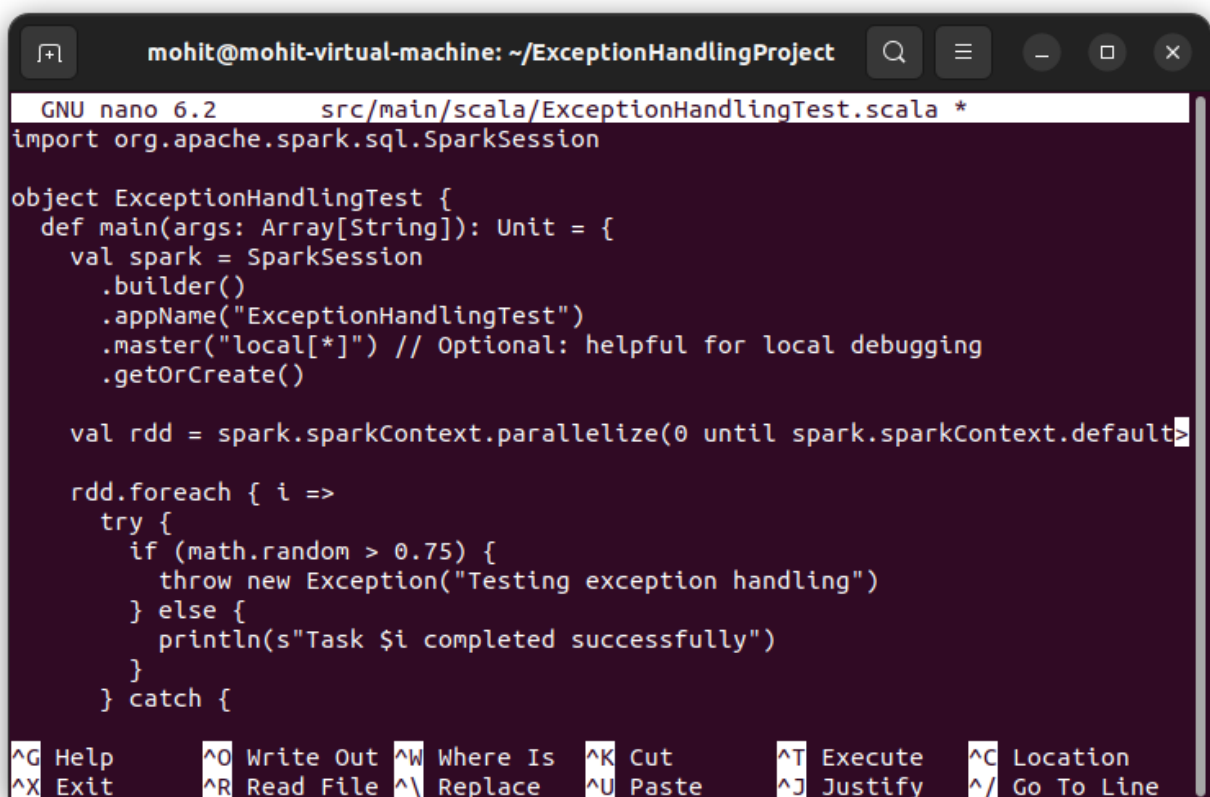


```

    }
    } catch { case e: Exception =>
println(s"Exception in task $i: ${e.getMessage}")
    }
}

spark.stop()
}
}

```



The screenshot shows a terminal window titled "mohit@mohit-virtual-machine: ~/ExceptionHandlingProject". The editor is GNU nano 6.2, editing the file "src/main/scala/ExceptionHandlingTest.scala". The code in the file is as follows:

```

import org.apache.spark.sql.SparkSession

object ExceptionHandlingTest {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession
      .builder()
      .appName("ExceptionHandlingTest")
      .master("local[*]") // Optional: helpful for local debugging
      .getOrCreate()

    val rdd = spark.sparkContext.parallelize(0 until spark.sparkContext.defaultParallelism)

    rdd.foreach { i =>
      try {
        if (math.random > 0.75) {
          throw new Exception("Testing exception handling")
        } else {
          println(s"Task $i completed successfully")
        }
      } catch {
        _ => {}
      }
    }
  }
}

```

At the bottom of the terminal, there is a status bar with various shortcuts: ^G Help, ^O Write Out, ^W Where Is, ^K Cut, ^T Execute, ^C Location, ^X Exit, ^R Read File, ^\ Replace, ^U Paste, ^J Justify, ^_ Go To Line.

Start Spark Master and Worker

```
$SPARK_HOME/sbin/start-master.sh
```

```
$SPARK_HOME/sbin/start-worker.sh spark://localhost:7077
```



```

mohit@mohit-virtual-machine:~$ $SPARK_HOME/sbin/start-master.sh
starting org.apache.spark.deploy.master.Master, logging to /home/mohit/spark/log
s/spark-mohit-org.apache.spark.deploy.master.Master-1-mohit-virtual-machine.out
mohit@mohit-virtual-machine:~$ $SPARK_HOME/sbin/start-worker.sh spark://localhost:7077
starting org.apache.spark.deploy.worker.Worker, logging to /home/mohit/spark/log
s/spark-mohit-org.apache.spark.deploy.worker.Worker-1-mohit-virtual-machine.out
mohit@mohit-virtual-machine:~$

```

Build the Scala project

```

mohit@mohit-virtual-machine:~$ ls
data.txt      exceptionhandlingtest.sbt    Public      Videos
Desktop       ExceptionHandlingTest.scala  snap        wordcountclasses
Documents     Music                        spark       wordcountproblem
Downloads     Pictures                     Templates   wordcountproblem.jar
mohit@mohit-virtual-machine:~$

```

```

mkdir ~/ExceptionHandlingProject cd
~/ExceptionHandlingProject mkdir -p
src/main/scala mv ~/ExceptionHandlingTest.scala
src/main/scala/ nano build.sbt

```

```

mohit@mohit-virtual-machine:~$
mohit@mohit-virtual-machine:~$ mkdir ~/ExceptionHandlingProject
cd ~/ExceptionHandlingProject
mohit@mohit-virtual-machine:~/ExceptionHandlingProject$ mkdir -p src/main/scala
mohit@mohit-virtual-machine:~/ExceptionHandlingProject$ mv ~/ExceptionHandlingTest.scala src/main/scala/
mohit@mohit-virtual-machine:~/ExceptionHandlingProject$ nano build.sbt

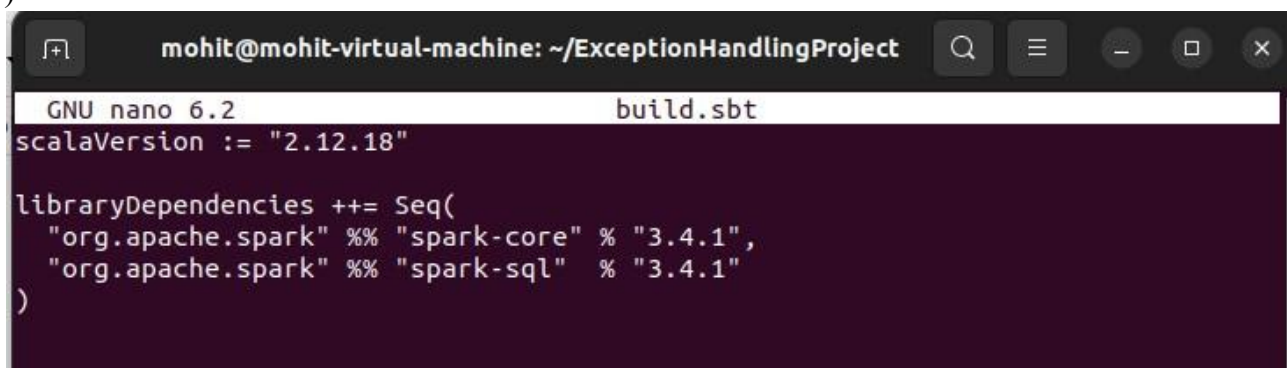
```

```
scalaVersion := "2.12.18"
```

```

libraryDependencies ++= Seq(
  "org.apache.spark" %% "spark-core" % "3.4.1",
  "org.apache.spark" %% "spark-sql" % "3.4.1"
)

```



```

GNU nano 6.2 build.sbt
scalaVersion := "2.12.18"

libraryDependencies ++= Seq(
  "org.apache.spark" %% "spark-core" % "3.4.1",
  "org.apache.spark" %% "spark-sql" % "3.4.1"
)

```

sbt package

Submit your program to Spark spark-

submit \

--class ExceptionHandlingTest \

--master local[*] \

target/scala-2.12/exceptionhandlingproject_2.12-0.1.0-SNAPSHOT.jar

```
mohit@mohit-virtual-machine:~/ExceptionHandlingProject$ spark-submit \
  --class ExceptionHandlingTest \
  --master local[*] \
  target/scala-2.12/exceptionhandlingproject_2.12-0.1.0-SNAPSHOT.jar
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
25/06/11 17:40:55 WARN Utils: Your hostname, mohit-virtual-machine, resolves to
a loopback address: 127.0.1.1; using 192.168.211.137 instead (on interface ens33
)
25/06/11 17:40:55 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
```

Practical No – 6

Aim: Write a Spark code to Handle the Streaming of data.

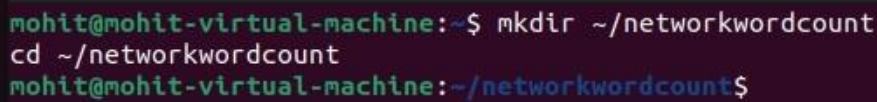
Introduction:

Spark Streaming enables scalable, high-throughput, fault-tolerant stream processing of live data streams. It divides the live data stream into **mini-batches**, performs **RDD** transformations, and outputs results in near real-time. Spark Streaming uses the same **Spark Core API**, which simplifies implementation and integration with batch and streaming workloads (Lambda Architecture).

Step 1: Project Setup (on Hadoop user)

```
mkdir ~/networkwordcount
```

```
cd ~/networkwordcount
```



```
mohit@mohit-virtual-machine:~$ mkdir ~/networkwordcount
cd ~/networkwordcount
mohit@mohit-virtual-machine:~/networkwordcount$
```

```
nano NetworkWordCount.scala
```

```
import org.apache.spark.SparkConf
```

```
import org.apache.spark.streaming._
```

```
object NetworkWordCount { def
```

```
main(args: Array[String]): Unit = {
```

```
    val sparkConf = new SparkConf()
```

```
    .setAppName("NetworkWordCount")
```

```
    .setMaster("local[2]")    val ssc = new
```

```
StreamingContext(sparkConf, Seconds(10))    val lines =
```

```
ssc.socketTextStream("localhost", 9999)
```

```
    val words = lines.flatMap(_.split(" "))    val
```

```
wordPairs = words.map(word => (word, 1))    val
```

```
wordCounts = wordPairs.reduceByKey(_ + _)
```

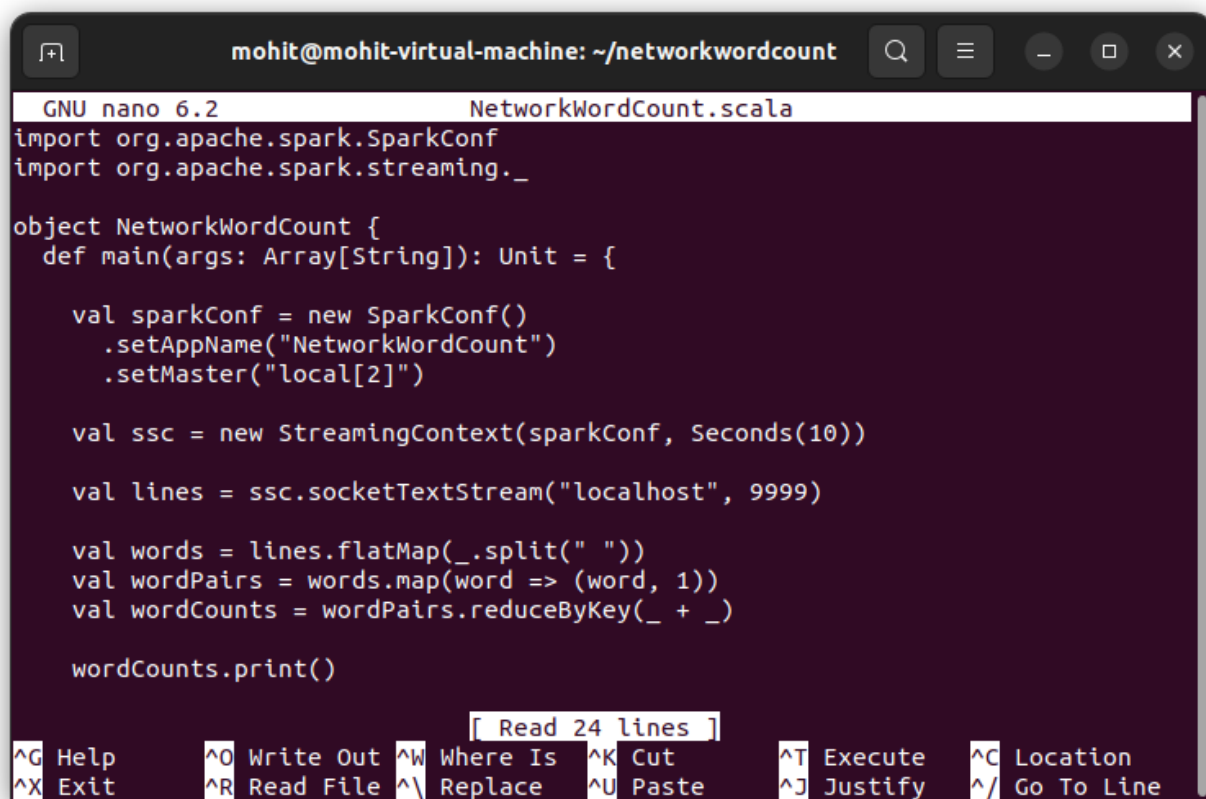
```
wordCounts.print()
```

```
    ssc.start()
```

```
    ssc.awaitTermination()
```

```
  }
```

```
}
```



```
GNU nano 6.2 NetworkWordCount.scala
import org.apache.spark.SparkConf
import org.apache.spark.streaming._

object NetworkWordCount {
  def main(args: Array[String]): Unit = {

    val sparkConf = new SparkConf()
      .setAppName("NetworkWordCount")
      .setMaster("local[2]")

    val ssc = new StreamingContext(sparkConf, Seconds(10))

    val lines = ssc.socketTextStream("localhost", 9999)

    val words = lines.flatMap(_.split(" "))
    val wordPairs = words.map(word => (word, 1))
    val wordCounts = wordPairs.reduceByKey(_ + _)

    wordCounts.print()
  }
}
```

[Read 24 lines]

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line

nano build.sbt name :=

"networkwordcount" version

:= "1.0.0" scalaVersion :=

"2.12.18"

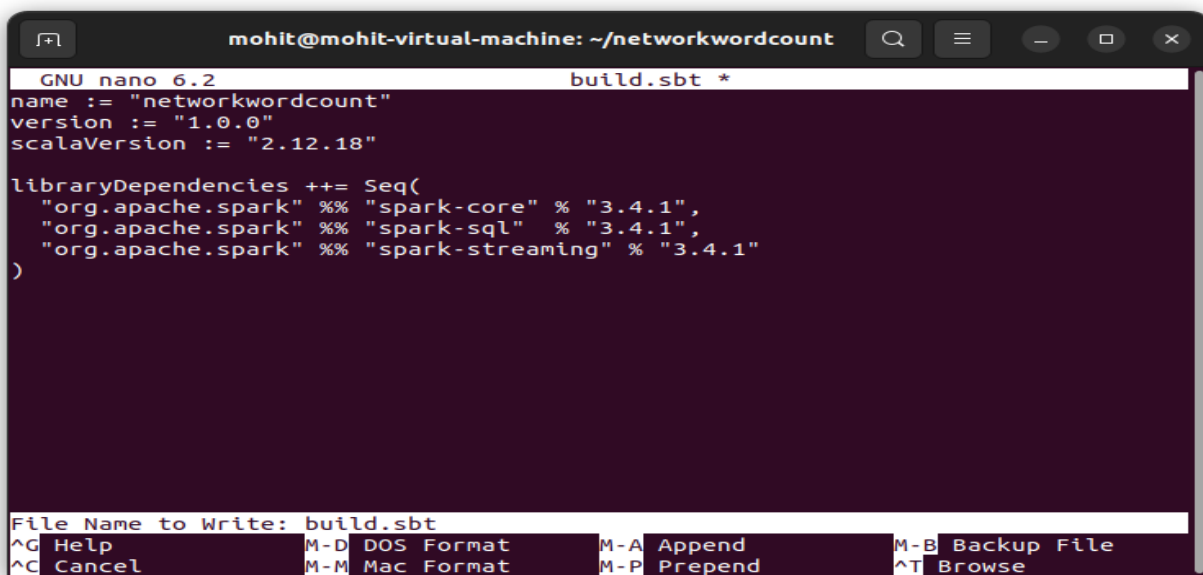
libraryDependencies += Seq(

"org.apache.spark" %% "spark-core" % "3.4.1",

"org.apache.spark" %% "spark-sql" % "3.4.1",

"org.apache.spark" %% "spark-streaming" % "3.4.1"

)



```

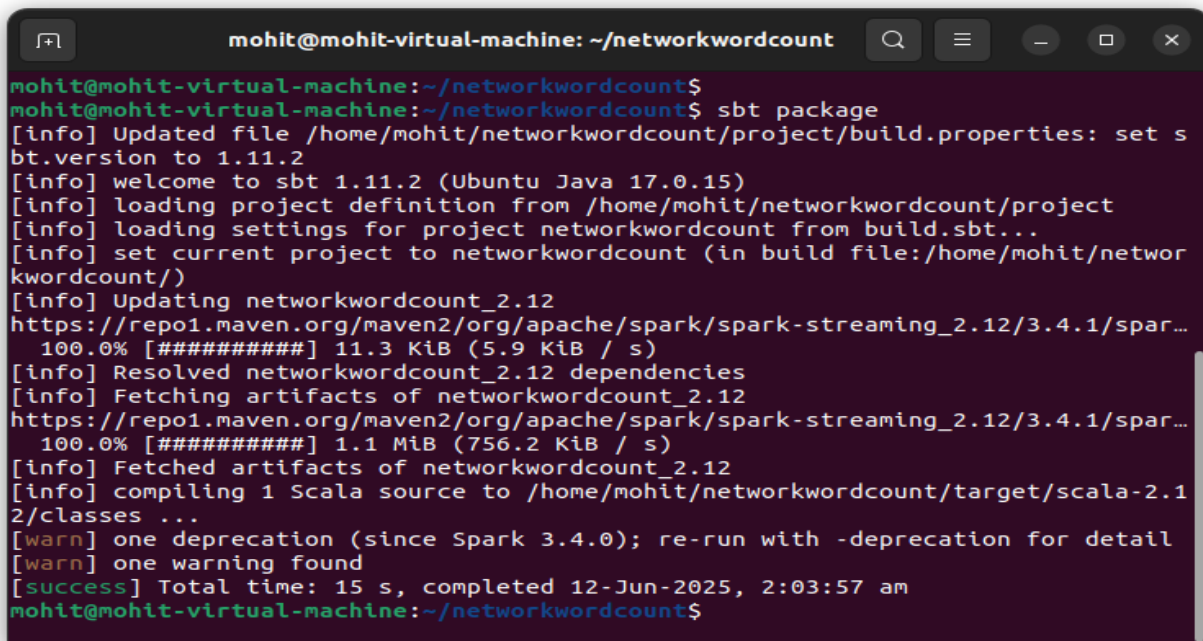
GNU nano 6.2 build.sbt *
name := "networkwordcount"
version := "1.0.0"
scalaVersion := "2.12.18"

libraryDependencies ++= Seq(
  "org.apache.spark" %% "spark-core" % "3.4.1",
  "org.apache.spark" %% "spark-sql" % "3.4.1",
  "org.apache.spark" %% "spark-streaming" % "3.4.1"
)

File Name to Write: build.sbt
^G Help          M-D DOS Format   M-A Append       M-B Backup File
^C Cancel        M-M Mac Format   M-P Prepend      ^T Browse

```

sbt package



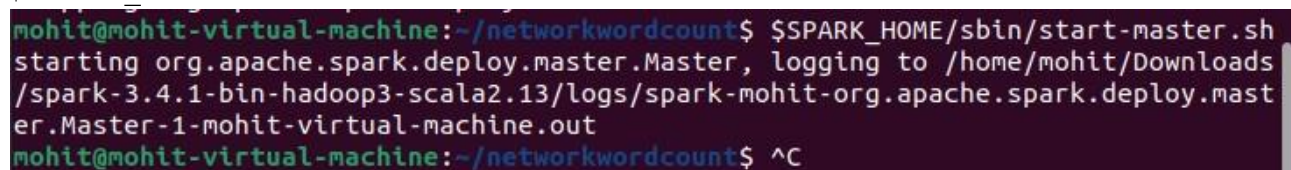
```

mohit@mohit-virtual-machine: ~/networkwordcount
mohit@mohit-virtual-machine:~/networkwordcount$ sbt package
[info] Updated file /home/mohit/networkwordcount/project/build.properties: set s
bt.version to 1.11.2
[info] welcome to sbt 1.11.2 (Ubuntu Java 17.0.15)
[info] loading project definition from /home/mohit/networkwordcount/project
[info] loading settings for project networkwordcount from build.sbt...
[info] set current project to networkwordcount (in build file:/home/mohit/networ
kwordcount/)
[info] Updating networkwordcount_2.12
https://repo1.maven.org/maven2/org/apache/spark/spark-streaming_2.12/3.4.1/spar...
100.0% [#####] 11.3 KiB (5.9 KiB / s)
[info] Resolved networkwordcount_2.12 dependencies
[info] Fetching artifacts of networkwordcount_2.12
https://repo1.maven.org/maven2/org/apache/spark/spark-streaming_2.12/3.4.1/spar...
100.0% [#####] 1.1 MiB (756.2 KiB / s)
[info] Fetched artifacts of networkwordcount_2.12
[info] compiling 1 Scala source to /home/mohit/networkwordcount/target/scala-2.1
2/classes ...
[warn] one deprecation (since Spark 3.4.0); re-run with -deprecation for detail
[warn] one warning found
[success] Total time: 15 s, completed 12-Jun-2025, 2:03:57 am
mohit@mohit-virtual-machine:~/networkwordcount$

```

Start the spark server

`$SPARK_HOME/sbin/start-master.sh`



```

mohit@mohit-virtual-machine: ~/networkwordcount$ $SPARK_HOME/sbin/start-master.sh
starting org.apache.spark.deploy.master.Master, logging to /home/mohit/Downloads
/spark-3.4.1-bin-hadoop3-scala2.13/logs/spark-mohit-org.apache.spark.deploy.mast
er.Master-1-mohit-virtual-machine.out
mohit@mohit-virtual-machine:~/networkwordcount$ ^C

```


`$SPARK_HOME/sbin/start-worker.sh spark://mohit-virtual-machine:7077`

```
mohit@mohit-virtual-machine:~/networkwordcount$ $SPARK_HOME/sbin/start-worker.sh
spark://mohit-virtual-machine:7077
starting org.apache.spark.deploy.worker.Worker, logging to /home/mohit/Downloads
/spark-3.4.1-bin-hadoop3-scala2.13/logs/spark-mohit-org.apache.spark.deploy.work
er.Worker-1-mohit-virtual-machine.out
```

`spark-submit \`

`--class NetworkWordCount \`

`--master spark://mohit-virtual-machine:7077 \ target/scala-2.12/networkwordcount_2.12-1.0.0.jar`

```
mohit@mohit-virtual-machine:~/networkwordcount$ spark-submit --class NetworkWord
Count \
--master spark://mohit-virtual-machine:7077 \
target/scala-2.12/networkwordcount_2.12-1.0.0.jar
25/06/12 17:42:08 WARN Utils: Your hostname, mohit-virtual-machine resolves to a
loopback address: 127.0.1.1; using 192.168.211.137 instead (on interface ens33)
25/06/12 17:42:08 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
25/06/12 17:42:09 INFO SparkContext: Running Spark version 3.4.1
25/06/12 17:42:09 WARN NativeCodeLoader: Unable to load native-hadoop library fo
r your platform... using builtin-java classes where applicable
25/06/12 17:42:09 INFO ResourceUtils: =====
=====
25/06/12 17:42:09 INFO ResourceUtils: No custom resources configured for spark.d
river.
25/06/12 17:42:09 INFO ResourceUtils: =====
=====
```

On a separate terminal, start the netscape server

```
mohit@mohit-virtual-machine:~$ nc -lk 9999

hello world
spark streaming is awesome
hello hello
```

OR

open the spark shell import

`org.apache.spark.streaming._ val ssc = new`

`StreamingContext(sc, Seconds(5))`

`val lines = ssc.socketTextStream("localhost", 9999) val words =`
`lines.flatMap(_.split(" ")) val wordCounts = words.map(word =>`
`(word, 1)).reduceByKey(_ + _) wordCounts.print()`

`ssc.start()`

ssc.awaitTermination()

```

mohit@mohit-virtual-machine: ~
scala> import org.apache.spark.streaming._
import org.apache.spark.streaming._

scala> val ssc = new StreamingContext(sc, Seconds(10))
warning: 1 deprecation (since Spark 3.4.0); for details, enable `:setting -depre
cation` or `:replay -deprecation`
val ssc: org.apache.spark.streaming.StreamingContext = org.apache.spark.streamin
g.StreamingContext@946b9d9

scala> val lines = ssc.socketTextStream("localhost", 9999)
| val words = lines.flatMap(_.split(" "))
| val wordCounts = words.map(word => (word, 1)).reduceByKey(_ + _)
val lines: org.apache.spark.streaming.dstream.ReceiverInputDStream[String] = org
.apache.spark.streaming.dstream.SocketInputDStream@772f3774
val words: org.apache.spark.streaming.dstream.DStream[String] = org.apache.spark
.streaming.dstream.FlatMappedDStream@6166d18b
val wordCounts: org.apache.spark.streaming.dstream.DStream[(String, Int)] = org.
apache.spark.streaming.dstream.ShuffledDStream@4270a134

scala> wordCounts.print()

scala> ssc.start()
| ssc.awaitTermination()
25/06/12 18:10:05 WARN RandomBlockReplicationPolicy: Expecting 1 replicas with o
nly 0 peer/s.
25/06/12 18:10:05 WARN BlockManager: Block input-0-1749732005400 replicated to o
nly 0 peer(s) instead of 1 peers
-----

```

```

mohit@mohit-virtual-machine: ~/networkwordcount$ nc -lk 9999
mohit verma
hello world i am learning spark

hello spark
spark is great
hello again

```

```

-----
Time: 1749732130000 ms
-----

```

```

(is,1)
(,2)
(hello,2)
(again,1)
(spark,2)
(great,1)
-----

```

```

Time: 1749732140000 ms
-----

```

Practical No – 7

Aim: Install HBase and use the HBase Data model Store and retrieve data

Introduction:

HBase is a **column-oriented, non-relational database**. This means that data is stored in individual columns, and indexed by a unique row key. This architecture allows for rapid retrieval of individual rows and columns and efficient scans over individual columns within a table.

Step1:

To work with HBase, we need to install Hadoop and the Java dependencies first. This was done in Practical One

start Hadoop start-

dfs.sh start-yarn.sh

```
hadoop@mohit-virtual-machine:~$ start-dfs.sh
Starting namenodes on [0.0.0.0]
0.0.0.0: namenode is running as process 8996. Stop it first.
Starting datanodes
localhost: datanode is running as process 9264. Stop it first.
Starting secondary namenodes [mohit-virtual-machine]
mohit-virtual-machine: secondarynamenode is running as process 9508. Stop it first.
hadoop@mohit-virtual-machine:~$ start-yarn.sh
Starting resourcemanager
resourcemanager is running as process 9707. Stop it first.
Starting nodemanagers
localhost: nodemanager is running as process 9965. Stop it first.
hadoop@mohit-virtual-machine:~$
```

Step 2:

Download Hbase

wget <https://archive.apache.org/dist/hbase/2.4.1/hbase-2.4.1-bin.tar.gz>

```
hadoop@mohit-virtual-machine:~$ wget https://archive.apache.org/dist/hbase/2.4.1/hbase-2.4.1-bin.tar.gz
--2025-07-03 09:10:49-- https://archive.apache.org/dist/hbase/2.4.1/hbase-2.4.1-bin.tar.gz
Resolving archive.apache.org (archive.apache.org)... 65.108.204.189, 2a01:4f9:1a:a084::2
```

Unzip it by executing command

tar -xvf hbase-2.4.1-bin.tar.gz

```
hadoop@mohit-virtual-machine:~$  
hadoop@mohit-virtual-machine:~$ tar -xvf hbase-2.4.1-bin.tar.gz  
hbase-2.4.1/LICENSE.txt  
hbase-2.4.1/NOTICE.txt  
hbase-2.4.1/LEGAL  
hbase-2.4.1/docs/
```

Rename directory to hbase

```
mv hbase-2.4.1 hbase
```

```
hadoop@mohit-virtual-machine:~$  
hadoop@mohit-virtual-machine:~$  
hadoop@mohit-virtual-machine:~$ mv hbase-2.4.1 hbase
```

It will unzip the contents, and it will create hbase-2.4.15 directory in the location /home/Hadoop

Now rename the directory to hbase

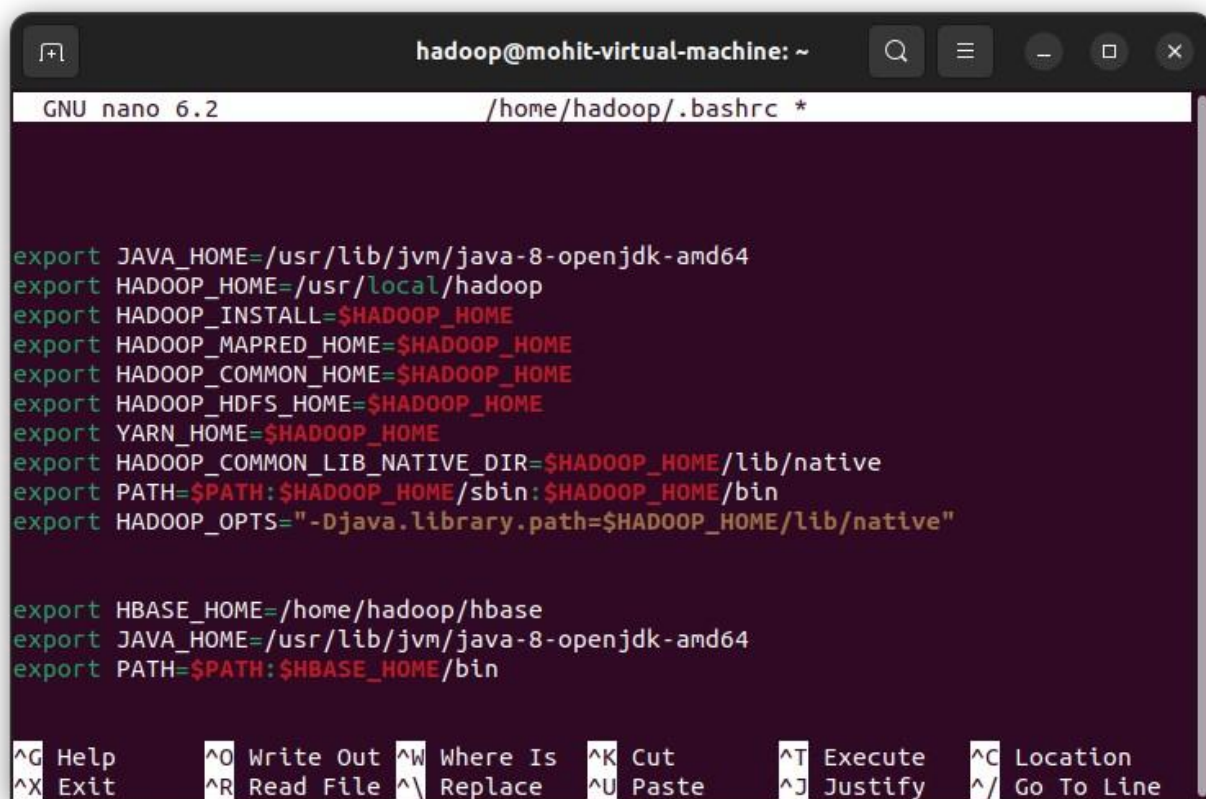
Set variables in bashc.

```
nano ~/.bashrc export
```

```
HBASE_HOME=/home/hadoop/hbase export
```

```
JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64 export
```

```
PATH=$PATH:$HBASE_HOME/bin
```

```
hadoop@mohit-virtual-machine: ~  
GNU nano 6.2 /home/hadoop/.bashrc *  
  
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64  
export HADOOP_HOME=/usr/local/hadoop  
export HADOOP_INSTALL=$HADOOP_HOME  
export HADOOP_MAPRED_HOME=$HADOOP_HOME  
export HADOOP_COMMON_HOME=$HADOOP_HOME  
export HADOOP_HDFS_HOME=$HADOOP_HOME  
export YARN_HOME=$HADOOP_HOME  
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native  
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin  
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"  
  
export HBASE_HOME=/home/hadoop/hbase  
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64  
export PATH=$PATH:$HBASE_HOME/bin  
  
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  ^C Location  
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^_ Go To Line
```

Read the edited bashrc file to the running memory source

~/.bashrc

```
hadoop@mohit-virtual-machine:~$ nano ~/.bashrc  
hadoop@mohit-virtual-machine:~$  
hadoop@mohit-virtual-machine:~$ source ~/.bashrc
```

Configure HBase:

nano ~/hbase/conf/hbase-env.sh export

JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64


```

GNU nano 6.2 /home/hadoop/hbase/conf/hbase-env.sh *
# The default log rolling policy is RFA, where the log file is rolled as per th>
# RFA appender. Please refer to the log4j.properties file to see more details o>
# In case one needs to do log rolling on a date change, one should set the envi>
# HBASE_ROOT_LOGGER to "<DESIRED_LOG LEVEL>,DRFA".
# For example:
# HBASE_ROOT_LOGGER=INFO,DRFA
# The reason for changing default to RFA is to avoid the boundary case of filli>
# DRFA doesn't put any cap on the log size. Please refer to HBase-5655 for more>

# Tell HBase whether it should include Hadoop's lib when start up,
# the default value is false, means that includes Hadoop's lib.
# export HBASE_DISABLE_HADOOP_CLASSPATH_LOOKUP="true"

# Override text processing tools for use by these launch scripts.
# export GREP="{GREP-grep}"
# export SED="{SED-sed}"

export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^_ Go To Line

```

Open HBase-site.xml and mention the below properties in the file.

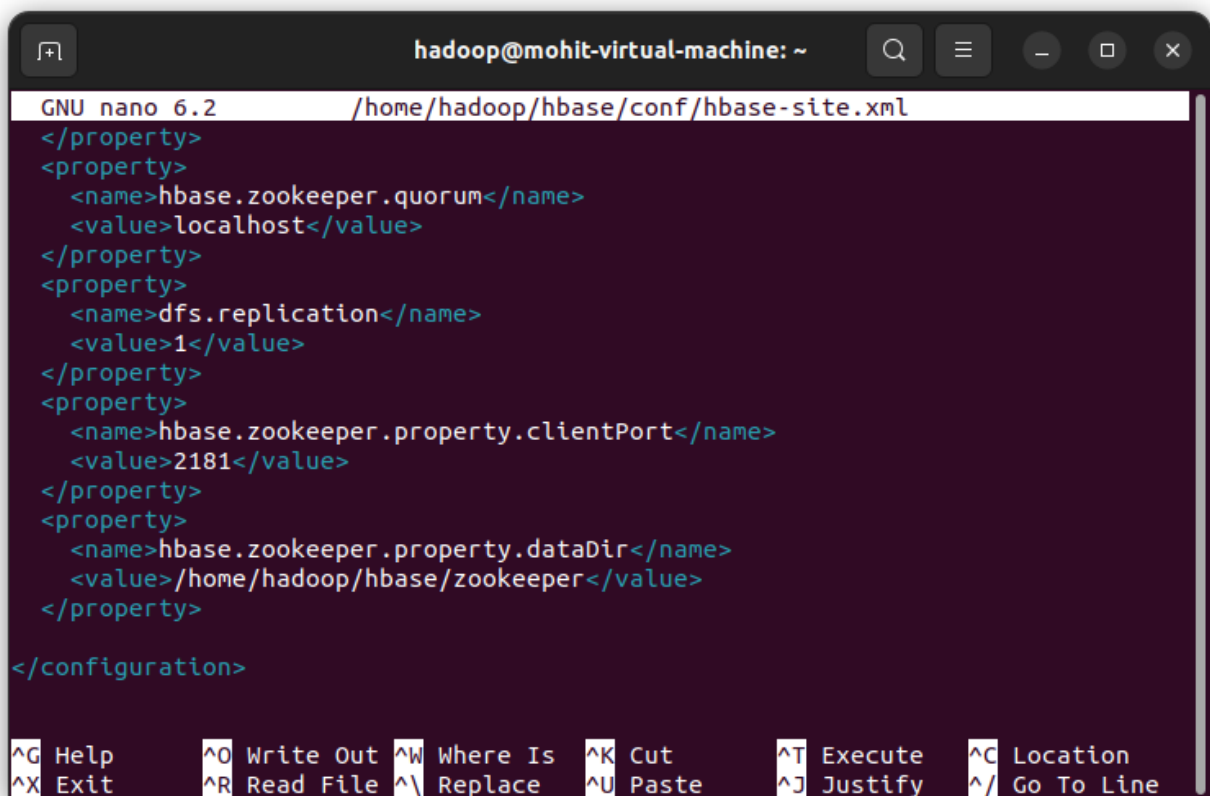
`nano ~/hbase/conf/hbase-site.xml`

```

<configuration>
  <property>
    <name>hbase.rootdir</name>
    <value>hdfs://localhost:9000/hbase</value>
  </property>
  <property>
    <name>hbase.cluster.distributed</name>
    <value>true</value>
  </property>
  <property>
    <name>hbase.zookeeper.quorum</name>
    <value>localhost</value>
  </property>

```

```
<property>
  <name>dfs.replication</name>
  <value>1</value>
</property>
<property>
  <name>hbase.zookeeper.property.clientPort</name>
  <value>2181</value>
</property>
<property>
  <name>hbase.zookeeper.property.dataDir</name>
  <value>/home/hadoop/hbase/zookeeper</value>
</property>
</configuration>
```



The screenshot shows a terminal window titled 'hadoop@mohit-virtual-machine: ~'. The window displays the nano 6.2 editor editing the file '/home/hadoop/hbase/conf/hbase-site.xml'. The content of the file is as follows:

```
</property>
<property>
  <name>hbase.zookeeper.quorum</name>
  <value>localhost</value>
</property>
<property>
  <name>dfs.replication</name>
  <value>1</value>
</property>
<property>
  <name>hbase.zookeeper.property.clientPort</name>
  <value>2181</value>
</property>
<property>
  <name>hbase.zookeeper.property.dataDir</name>
  <value>/home/hadoop/hbase/zookeeper</value>
</property>
</configuration>
```

At the bottom of the terminal, there is a status bar with various keyboard shortcuts: ^G Help, ^O Write Out, ^W Where Is, ^K Cut, ^T Execute, ^C Location, ^X Exit, ^R Read File, ^_ Replace, ^U Paste, ^J Justify, and ^_/ Go To Line.

Properties Explanation

1. Setting up Hbase root directory in this property, for distributed set up we have to set this property

2. ZooKeeper quorum property should be set up here
3. Replication set up done in this property. By default we are placing replication as 1. In the fully distributed mode, multiple data nodes present so we can increase replication by placing more than 1 value in the dfs.replication property
4. Client port should be mentioned in this property
5. ZooKeeper data directory can be mentioned in this property

Start Hadoop daemons first and after that start HBase daemons as shown below:

start-hbase.sh

```

hadoop@mohit-virtual-machine: ~$
hadoop@mohit-virtual-machine:~$ start-hbase.sh
/usr/local/hadoop/libexec/hadoop-functions.sh: line 2358: HADOOP_ORG.APACHE.HADO
OP.HBASE.UTIL.GETJAVAPROPERTY_USER: invalid variable name
/usr/local/hadoop/libexec/hadoop-functions.sh: line 2453: HADOOP_ORG.APACHE.HADO
OP.HBASE.UTIL.GETJAVAPROPERTY_OPTS: invalid variable name
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/local/hadoop/share/hadoop/common/lib/slf4
j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/home/hadoop/hbase/lib/client-facing-thirdpart
y/slf4j-log4j12-1.7.30.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
/usr/local/hadoop/libexec/hadoop-functions.sh: line 2358: HADOOP_ORG.APACHE.HADO
OP.HBASE.UTIL.GETJAVAPROPERTY_USER: invalid variable name
/usr/local/hadoop/libexec/hadoop-functions.sh: line 2453: HADOOP_ORG.APACHE.HADO
OP.HBASE.UTIL.GETJAVAPROPERTY_OPTS: invalid variable name
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/local/hadoop/share/hadoop/common/lib/slf4
j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/home/hadoop/hbase/lib/client-facing-thirdpart
y/slf4j-log4j12-1.7.30.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]

```

now open Hbase shell hbase

shell

```

hadoop@mohit-virtual-machine: ~
rty/slf4j-log4j12-1.7.30.jar!/org/slf4j/impl/StaticLoggerBinder.class]
: SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanatio
n.
hadoop@mohit-virtual-machine:~$ hbase shell
/usr/local/hadoop/libexec/hadoop-functions.sh: line 2358: HADOOP_ORG.APACHE.HADO
OP.HBASE.UTIL.GETJAVAPROPERTY_USER: invalid variable name
/usr/local/hadoop/libexec/hadoop-functions.sh: line 2453: HADOOP_ORG.APACHE.HADO
OP.HBASE.UTIL.GETJAVAPROPERTY_OPTS: invalid variable name
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/local/hadoop/share/hadoop/common/lib/slf4
j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/home/hadoop/hbase/lib/client-facing-thirdpart
y/slf4j-log4j12-1.7.30.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: http://hbase.apache.org/2.0/book.html#shell
Version 2.4.1, rb4d9639f66fccdb45fea0244202ffbd755341260, Fri Jan 15 10:58:57 PS
T 2021
Took 0.0035 seconds
hbase:001:0>
hbase:002:0>

```

Create an HBase table and define column families

create 'emp', 'pri_data', 'pro_data'

```

Took 0.0035 seconds
hbase:001:0>
hbase:002:0> create 'emp', 'pri_data', 'pro_data'

Created table emp
Took 1.6122 seconds
=> Hbase::Table - emp
hbase:003:0>
hbase:004:0>

```

Insert Sample Data (PUT)

```

put 'emp', '1', 'pri_data:name', 'Andy' put
'emp', '1', 'pri_data:age', '22' put 'emp', '1',
'pro_data:post', 'asst. manager' put 'emp', '1',
'pro_data:salary', '40k' put 'emp', '2',
'pri_data:name', 'Icarus'

```

```
put 'emp', '2', 'pri_data:age', '22' put
'emp', '2', 'pro_data:post', 'manager'
```

```
hbase:004:0>
hbase:005:0> put 'emp', '1', 'pri_data:name', 'Andy'
Took 0.1395 seconds
hbase:006:0> put 'emp', '1', 'pri_data:age', '22'
Took 0.0070 seconds
hbase:007:0> put 'emp', '1', 'pro_data:post', 'asst. manager'
Took 0.0169 seconds
hbase:008:0> put 'emp', '1', 'pro_data:salary', '40k'
Took 0.0045 seconds
hbase:009:0> put 'emp', '2', 'pri_data:name', 'Icarus'
Took 0.0054 seconds
hbase:010:0> put 'emp', '2', 'pri_data:age', '22'
Took 0.0076 seconds
hbase:011:0> put 'emp', '2', 'pro_data:post', 'manager'
Took 0.0075 seconds
hbase:012:0> █
```

Retrieve Data (GET)

```
get 'emp', '1'
get 'emp', '2'
```

```
hbase:013:0> get 'emp', '1'
COLUMN                                CELL
pri_data:age                          timestamp=2025-07-03T09:40:11.438, value=22
pri_data:name                          timestamp=2025-07-03T09:40:11.396, value=Andy
pro_data:post                          timestamp=2025-07-03T09:40:11.478, value=asst. manager
pro_data:salary                       timestamp=2025-07-03T09:40:11.513, value=40k
1 row(s)
Took 0.0594 seconds
hbase:014:0> get 'emp', '2'
COLUMN                                CELL
pri_data:age                          timestamp=2025-07-03T09:40:11.567, value=22
pri_data:name                          timestamp=2025-07-03T09:40:11.540, value=Icarus
pro_data:post                          timestamp=2025-07-03T09:40:11.595, value=manager
1 row(s)
Took 0.0059 seconds
hbase:015:0> █
```

Delete a Specific Cell (column-level delete)

```
delete 'emp', '1', 'pri_data:city'
```

```
hbase:015:0>
hbase:016:0> delete 'emp', '1', 'pri_data:city'
Took 0.0164 seconds
hbase:017:0> █
```

```
put 'emp', '1', 'pri_data:city', 'Delhi'
```



```
hbase:018:0>
hbase:019:0> put 'emp', '1', 'pri_data:city', 'Delhi'
Took 0.0057 seconds
hbase:020:0>
```

Delete Entire Row (row-level delete)

deleteall 'emp', '1'

```
hbase:020:0>
hbase:021:0> deleteall 'emp', '1'
Took 0.0115 seconds
hbase:022:0>
```

Scan All Rows scan

'emp'

```
hbase:022:0>
hbase:023:0> scan 'emp'
ROW          COLUMN+CELL
 2          column=pri_data:age, timestamp=2025-07-03T09:40:11.567, va
           lue=22
 2          column=pri_data:name, timestamp=2025-07-03T09:40:11.540, v
           alue=Icarus
 2          column=pro_data:post, timestamp=2025-07-03T09:40:11.595, v
           alue=manager
1 row(s)
Took 0.0175 seconds
hbase:024:0>
```

List All Tables

list

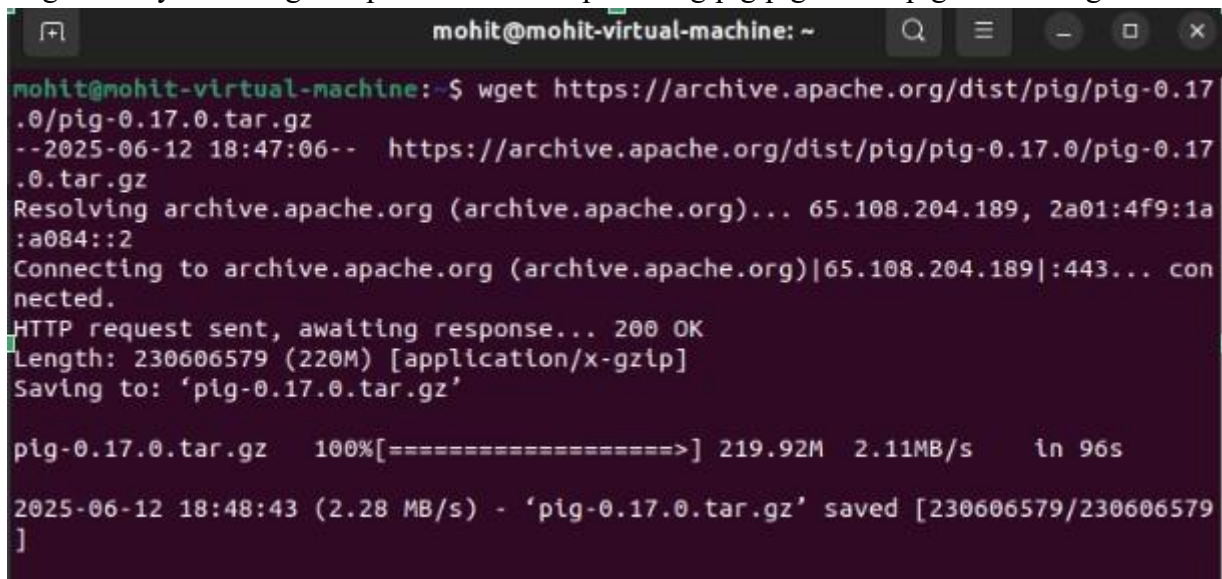
```
hbase:027:0>
hbase:028:0> list
TABLE
emp
1 row(s)
Took 0.0189 seconds
=> ["emp"]
hbase:029:0>
```

Practical No – 8

Aim: Write a Pig Script for solving counting problems.

Introduction:

Apache Pig is a high-level platform for creating programs that run on Apache Hadoop. The language for this platform is called Pig Latin. Pig can execute its Hadoop jobs in MapReduce, Apache Tez, or Apache Spark. Pig Latin abstracts the programming from the Java MapReduce idiom into a notation which makes MapReduce programming high level, similar to that of SQL for relational database management systems. `wget https://downloads.apache.org/pig/pig-0.17.0/pig-0.17.0.tar.gz`

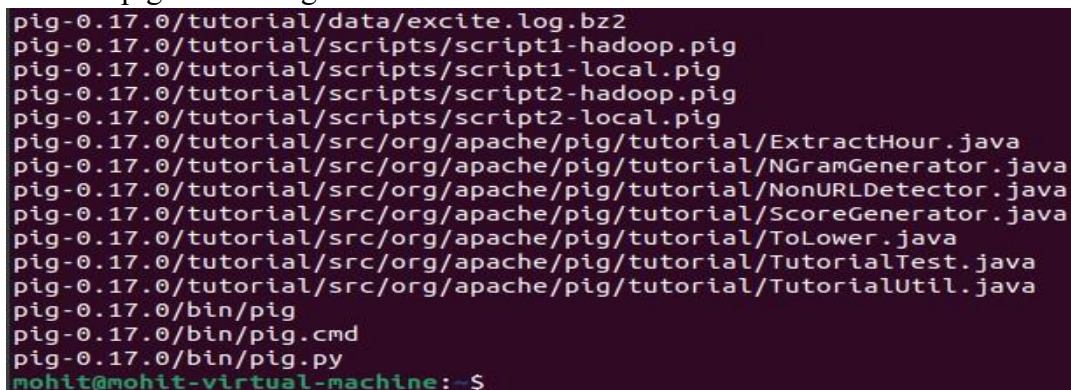


```
mohit@mohit-virtual-machine: ~
mohit@mohit-virtual-machine:~$ wget https://archive.apache.org/dist/pig/pig-0.17.0/pig-0.17.0.tar.gz
--2025-06-12 18:47:06-- https://archive.apache.org/dist/pig/pig-0.17.0/pig-0.17.0.tar.gz
Resolving archive.apache.org (archive.apache.org)... 65.108.204.189, 2a01:4f9:1a:a084::2
Connecting to archive.apache.org (archive.apache.org)[65.108.204.189]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 230606579 (220M) [application/x-gzip]
Saving to: 'pig-0.17.0.tar.gz'

pig-0.17.0.tar.gz  100%[=====>] 219.92M  2.11MB/s   in 96s

2025-06-12 18:48:43 (2.28 MB/s) - 'pig-0.17.0.tar.gz' saved [230606579/230606579]
```

`tar -xvzf pig-0.17.0.tar.gz`



```

pig-0.17.0/tutorial/data/excite.log.bz2
pig-0.17.0/tutorial/scripts/script1-hadoop.pig
pig-0.17.0/tutorial/scripts/script1-local.pig
pig-0.17.0/tutorial/scripts/script2-hadoop.pig
pig-0.17.0/tutorial/scripts/script2-local.pig
pig-0.17.0/tutorial/src/org/apache/pig/tutorial/ExtractHour.java
pig-0.17.0/tutorial/src/org/apache/pig/tutorial/NGramGenerator.java
pig-0.17.0/tutorial/src/org/apache/pig/tutorial/NonURLDetector.java
pig-0.17.0/tutorial/src/org/apache/pig/tutorial/ScoreGenerator.java
pig-0.17.0/tutorial/src/org/apache/pig/tutorial/ToLower.java
pig-0.17.0/tutorial/src/org/apache/pig/tutorial/TutorialTest.java
pig-0.17.0/tutorial/src/org/apache/pig/tutorial/TutorialUtil.java
pig-0.17.0/bin/pig
pig-0.17.0/bin/pig.cmd
pig-0.17.0/bin/pig.py
mohit@mohit-virtual-machine:~$
```

move your hadoop and set your java_home and pig environment

`sudo mv /home/hadoop/pig ~/ export`

`PIG_HOME=/home/mohit/pig`

`export PATH=$PATH:$PIG_HOME/bin export`

`PIG_CLASSPATH=$HADOOP_HOME/etc/hadoop`

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH
```

```
mohit@mohit-virtual-machine: ~$ sudo mv /home/hadoop/pig ~/
mohit@mohit-virtual-machine: ~$ nano ~/.bashrc
mohit@mohit-virtual-machine: ~$ source ~/.bashrc
```

```
export SPARK_HOME=~/.Downloads/spark-3.4.1-bin-hadoop3-scala2.13
export PATH=$SPARK_HOME/bin:$PATH
```

```
export PIG_HOME=/home/mohit/pig
export PATH=$PATH:$PIG_HOME/bin
export PIG_CLASSPATH=$HADOOP_HOME/etc/hadoop
```

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH
```

```
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

check the pig -version

pig -version

```
mohit@mohit-virtual-machine: ~$ pig -version
Apache Pig version 0.17.0 (r1797386)
compiled Jun 02 2017, 15:41:58
```

solving counting problems

create a txt file nano

/home/mohit/textfile.txt

hello world hello hadoop pig

is powerful

```
mohit@mohit-virtual-machine: ~
GNU nano 6.2 /home/mohit/textfile.txt
hello world
hello hadoop
pig is powerful
```

Screenshot captured
You can paste the image

open pig in local model

pig -x local

run this script for wordcount problem

```

lines = LOAD '/home/mohit/textfile.txt' AS (line:chararray); words =
FOREACH lines GENERATE FLATTEN(TOKENIZE(line)) as word;
grouped = GROUP words BY word;
wordcount = FOREACH grouped GENERATE group, COUNT(words);
DUMP wordcount;

```

```

grunt> lines = LOAD '/home/mohit/textfile.txt' AS (line:chararray);
2025-06-12 19:25:29,961 [main] INFO org.apache.hadoop.conf.Configuration.deprec
ation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum
grunt> words = FOREACH lines GENERATE FLATTEN(TOKENIZE(line)) as word;
2025-06-12 19:25:41,331 [main] INFO org.apache.pig.impl.util.SpillableMemoryMan
ager - Selected heap (G1 Old Gen) of size 1048576000 to monitor. collectionUsage
Threshold = 734003200, usageThreshold = 734003200
grunt> grouped = GROUP words BY word;
grunt> wordcount = FOREACH grouped GENERATE group, COUNT(words);
grunt> DUMP wordcount;

```

```

2025-06-12 19:26:09,248 [main] INFO org.apache.hadoop.mapreduce.reduce.InputFormat
inputFormat - Total input paths to process : 1
2025-06-12 19:26:09,248 [main] INFO org.apache.pig.backend.hadoop.executionengi
ne.util.MapRedUtil - Total input paths to process : 1
(is,1)
(pig,1)
(hello,2)
(world,1)
(hadoop,1)
(powerful,1)
grunt>

```